

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

СЫКТЫВКАРСКИЙ ЛЕСНОЙ ИНСТИТУТ (ФИЛИАЛ) ФЕДЕРАЛЬНОГО
ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО ОБРАЗОВАТЕЛЬНОГО
УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЛЕСОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМЕНИ С. М. КИРОВА» (СЛИ)

ТРАНСПОРТНО-ТЕХНОЛОГИЧЕСКИЙ ФАКУЛЬТЕТ

КАФЕДРА ИНФОРМАЦИОННЫХ СИСТЕМ И ТЕХНОЛОГИЙ

**ОТЧЁТ ПО ТЕХНОЛОГИЧЕСКОЙ (ПРОЕКТНО-ТЕХНОЛОГИЧЕСКОЙ)
ПРАКТИКЕ. WEB-ТЕХНОЛОГИИ**

на тему: Разработка CMS для парикмахерских и барбершопов «Сайтама»
Место прохождения практики: СЛИ
Период прохождения: 22.06.26 – 18.07.26

Выполнил: студент 3 курса
очной формы обучения

направления подготовки
бакалавриата «Информационные
системы и технологии»

_____ Решетняк В. А.
(подпись, дата)

Руководитель:
К. Э. Н., доцент

_____ Оганезова Н. А.
(подпись, дата)

Заведующий кафедрой
информационных систем и
технологий, к. ф.-м. н., доцент

_____ Плешев Д. А.
(подпись, дата)

Отчёт защищён на оценку _____

«_____» _____ 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 Постановка задачи	8
1.1 Цель и задачи разработки	8
1.2 Автоматизируемые бизнес-процессы	9
2 Архитектура программного обеспечения	11
2.1 Общая схема	11
2.2 Архитектурные шаблоны бэкенда	12
2.3 Архитектура фронтенда	13
2.4 Требования законодательства и защита данных	16
3 Обоснование выбора средств реализации	20
3.1 Метод экспертных оценок	20
3.2 Список альтернатив	20
3.3 Итоговые средства реализации	23
3.4 Дополнительные инструменты разработки	25
4 Структура базы данных	28
4.1 Инфологическая модель	28
4.2 Даталогическая модель	31
4.3 Индексы и оптимизация выборок	35
5 Реализация и развёртывание приложения	40
5.1 Общие сведения	40
5.2 Создание записи (бронирование)	40
5.3 Первый запуск системы (мастер /setup)	40
5.4 Интерфейс разработанного приложения	43
5.5 Подключение аналитики	48
5.6 Размещение в сети Интернет	49
ЗАКЛЮЧЕНИЕ	50
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	51
ПРИЛОЖЕНИЕ А. Диаграммы последовательностей	53
ПРИЛОЖЕНИЕ Б. Техническое задание	59

СПИСОК СОКРАЩЕНИЙ

API – Application Programming Interface – программный интерфейс взаимодействия компонентов системы.

CI – Continuous Integration – непрерывная автоматическая проверка (тесты, линтеры) при каждом изменении кода.

CMS – Content Management System – система управления содержимым сайта.

CRM – Customer Relationship Management – управление взаимоотношениями с клиентами.

CSP – Content Security Policy – политика безопасности контента браузера.

DTO – Data Transfer Object – объект переноса данных на границе API, отделённый от внутреннего представления в БД.

HTTP – HyperText Transfer Protocol – протокол передачи данных прикладного уровня; **HTTPS** – тот же протокол поверх зашифрованного канала TLS.

JSON – JavaScript Object Notation – текстовый формат обмена данными.

JWT – JSON Web Token – формат токена авторизации, используемый для access-токена сессии.

KPI – Key Performance Indicator – ключевой показатель эффективности.

MVCC – Multiversion Concurrency Control – многоверсионное управление конкурентным доступом в СУБД.

OAuth – открытый протокол делегированной авторизации через внешнего поставщика учётных записей.

ORM – Object-Relational Mapping – отображение объектов языка программирования на строки реляционной таблицы.

REST – Representational State Transfer – архитектурный стиль построения API поверх HTTP.

SEO – Search Engine Optimization – оптимизация страниц под поисковые системы.

SPA – Single Page Application – одностраничное веб-приложение

(фронтенд).

SQL – Structured Query Language – язык запросов к реляционной СУБД.

SSR – Server-Side Rendering – формирование страницы на стороне сервера.

TLS – Transport Layer Security – протокол шифрования канала связи (HTTPS).

UI – User Interface – пользовательский интерфейс.

VK ID – сервис единой учётной записи «ВКонтакте», используемый как внешний поставщик авторизации.

БД – база данных.

СУБД – система управления базами данных (в проекте – PostgreSQL 16).

ТЗ – техническое задание.

ФТ – функциональное требование (карточки ФТ-001–ФТ-016 в техническом задании).

СПИСОК ТЕРМИНОВ

Мастер – сотрудник точки, оказывающий услуги клиентам; в системе – роль пользователя.

Слот – свободный интервал в расписании мастера, доступный для записи клиента.

Точка сети – отдельная парикмахерская или барбершоп в составе сети одного владельца – собственный адрес, часы работы, мастера и записи.

ВВЕДЕНИЕ

Предприятия сферы парикмахерских услуг и барберинга всё чаще переходят от бумажных журналов и разрозненных таблиц к специализированным программным системам. Такие решения обеспечивают онлайн-запись клиентов, ведение расписания мастеров и автоматизацию отчётности. Отсутствие единого инструмента порождает типовые для отрасли проблемы: возникновение накладок в расписании (двойных записей), неявки клиентов из-за отсутствия напоминаний, сложности с дифференцированным учётом цен мастеров, а также отсутствие у руководства оперативной аналитики по выручке и загрузке заведений. Настоящий документ посвящён веб-приложению «Сайтама», автоматизирующему данные бизнес-процессы для отдельных салонов и сетевых барбершопов.

В работе формулируется решаемая задача, описываются автоматизируемые процессы, архитектура приложения, применённые паттерны проектирования и требования нормативно-правовой базы. Обосновывается выбор технологического стека (FastAPI [1], SQLAlchemy [2], PostgreSQL [3]), приводится схема базы данных, детально рассматривается логика ключевых пользовательских сценариев, интеграция аналитических модулей и развёртывание сервиса в сети Интернет.

Комплект проектной документации включает три сопутствующих документа:

- 1) Техническое задание, оформленное в соответствии с ГОСТ 34.602-2020 [4] и ГОСТ 19.201-78 [5], содержащее постановку задачи и перечень функциональных требований (ФТ-001–ФТ-016);
- 2) Руководство пользователя, описывающее пошаговые сценарии работы для четырёх ролей: клиента, мастера, администратора филиала и владельца сети;
- 3) Руководство администратора, регламентирующее процедуры развёртывания, эксплуатации и резервного копирования серверной инфраструктуры.

Практическая значимость результатов заключается в создании готовой к практическому применению программной платформы, устраняющей издержки ручного учёта и человеческого фактора. Архитектура системы

изначально поддерживает мультифилиальность, что позволяет развёртывать её как в единичных парикмахерских, так и в разветвлённых сетях без внесения изменений в исходный код.

1 Постановка задачи

1.1 Цель и задачи разработки

Большинство предприятий малого бизнеса в сфере парикмахерских услуг и барберинга ведут запись клиентов и операционный учёт вручную – посредством телефонных звонков, бумажных журналов и разрозненных электронных таблиц. Подобный подход порождает типовые для отрасли проблемы: возникновение накладок в расписании (двойных бронирований), высокий процент неявок клиентов (no-show) из-за отсутствия автоматических напоминаний, сложности с дифференцированным расчётом стоимости услуг по мастерам, а также невозможность оперативного получения сводной аналитики по выручке и загрузке заведений без трудоёмкого ручного подсчёта.

Основной целью работы является проектирование и разработка веб-приложения «Сайтама», автоматизирующего полный цикл операционной деятельности парикмахерской или барбершопа: организацию самостоятельной онлайн-записи клиентов без участия администратора, ведение каталога услуг с индивидуальным ценообразованием, отправку автоматических напоминаний о предстоящих визитах, а также генерацию отчётности по ключевым бизнес-показателям. Архитектура разрабатываемой системы изначально предусматривает поддержку мультифилиальности с разграничением прав доступа между администратором конкретного филиала и владельцем всей сети заведений.

Достижение поставленной цели оценивается следующими измеримыми показателями (детализированными в подразделе 2.2 технического задания):

- сокращение среднего времени оформления записи со стороны клиента до 2 минут;
- полное исключение вероятности двойных записей программно-техническими средствами за счёт ограничений целостности СУБД, а не административных регламентов;
- снижение доли неявок клиентов за счёт внедрения автоматических уведомлений;

- реализация гибкой настройки прайс-листа с учётом квалификации конкретного мастера;
- консолидация данных в едином реляционном хранилище и отказ от бумажного документооборота;
- обеспечение возможности самостоятельного формирования отчётов по выручке и загруженности заведений конечными пользователями без привлечения технических специалистов.

Границы проекта строго зафиксированы: за рамками разработки оставлены модули интернет-эквайринга (расчёты производятся очно на месте оказания услуг согласно подразделу 3.2 технического задания), ведение складского и бухгалтерского учёта, а также расширенные маркетинговые инструменты CRM-системы (сегментация клиентской базы, массовые рассылки и программы лояльности).

1.2 Автоматизируемые бизнес-процессы

Система автоматизирует следующие шесть ключевых бизнес-процессов, выделенных при обследовании предметной области (техническое задание, подраздел 2.1):

- 1) **Онлайн-запись клиентов (Зап-1)** – выбор филиала, услуги, мастера и свободного временного слота с последующим подтверждением записи без участия администратора. Для новых клиентов процедура бронирования предусматривает автоматическое создание учётной записи.
- 2) **Управление каталогом услуг и ценообразованием (Усл-2)** – ведение номенклатуры услуг с возможностью гибкой настройки стоимости для каждой связки «мастер – услуга».
- 3) **Оповещение и напоминания о визитах (Увед-3)** – автоматическая отправка уведомлений клиентам за 24 и за 2 часа до назначенного времени приёма.
- 4) **Формирование аналитической отчётности (Отчёт-4)** – расчёт показателей выручки, загруженности мастеров и рейтинга востребованности услуг за произвольный период в пользовательском интерфейсе без необходимости прямого обращения к базе данных.
- 5) **Управление контентом витрины (Конт-5)** – администриро-

вание информационной составляющей публичной части сервиса (главная страница, сведения о мастерах и их портфолио).

6) **Управление филиальной сетью (Точ-6)** – создание, редактирование и вывод из эксплуатации точек обслуживания с настройкой их адресов, контактных данных, графиков работы и разграничением прав доступа между локальными администраторами и владельцем сети (руководство пользователя, подраздел 4.4).

2 Архитектура программного обеспечения

2.1 Общая схема

Разрабатываемая система представляет собой клиент-серверное веб-приложение, состоящее из одностраничного клиентского интерфейса (SPA) на базе фреймворка Vue 3 [6] и серверной части с REST API на FastAPI [1], взаимодействующей с централизованной реляционной СУБД PostgreSQL [3].

Слоистая архитектура бэкенда и схема его взаимодействия с клиентским приложением представлены в соответствии с рисунком 1. Топология развёртывания компонентов системы (контейнеризация на базе Docker, конфигурация обратного прокси-сервера и настройка протокола TLS) детально приведена в руководстве администратора (раздел 1).

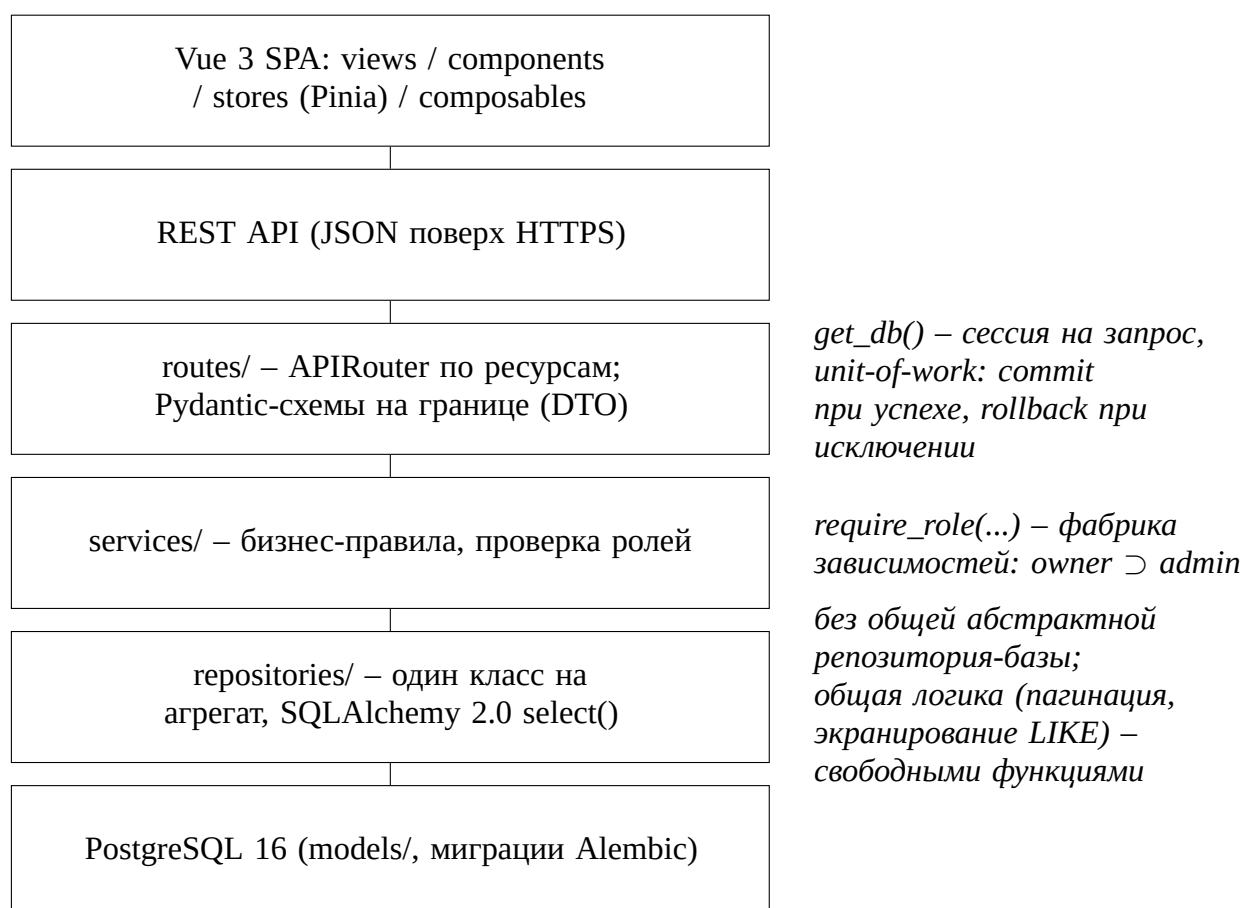


Рисунок 1 – Слоистая архитектура: от фронтенда до базы данных

2.2 Архитектурные шаблоны бэкенда

Серверная часть приложения (backend/app/) реализована на основе строгой однонаправленной слоистой архитектуры, включающей уровни маршрутизации (routes), бизнес-логики (services), доступа к данным (repositories) и объектно-реляционных моделей (models).

Вспомогательный пакет schemas/, содержащий Pydantic-схемы [7], применяется исключительно на границе обработки входящих и исходящих HTTP-запросов. Сервисный слой выполняет валидацию и сериализацию данных через вызов `XResponse.model_validate(orm_obj)`, что исключает прямую передачу внутренних ORM-сущностей во внешнее API.

В архитектуре серверной части применены следующие паттерны и подходы к проектированию:

- **Объект переноса данных (Data Transfer Object, DTO)** – разделение контрактов API (schemas/) и структур хранения базы данных (models/) обеспечивает независимость публичного интерфейса от изменений внутренней схемы данных;

- **Репозиторий (Repository)** – инкапсуляция логики взаимодействия с СУБД через SQLAlchemy [2] в специализированных классах (UserRepository, AppointmentRepository, SalonRepository). Слой бизнес-логики изолирован от формирования прямых SQL-запросов;

- **Внедрение зависимостей (Dependency Injection) и единица работы (Unit of Work)** – передача зависимостей осуществляется через встроенный механизм Depends фреймворка FastAPI. Генератор сессий `get_db()` реализует паттерн Unit of Work: в рамках одного HTTP-запроса выполняется единая транзакция с автоматической фиксацией изменений (`commit`) при успешном ответе, откатом (`rollback`) при возникновении исключений и гарантированным закрытием сессии (`close`) в блоке `finally`;

- **Фабрика зависимостей (Dependency Factory)** – параметризованная функция контроля доступа `require_role(*roles)` генерирует функции-зависимости для аутентификации пользователей (`get_current_client`, `get_current_master`, `get_current_admin`, `get_current_owner`). Поскольку права роли `owner` включают в себя все привилегии роли `admin`, проверка `get_current_admin` предоставляет доступ обеим категориям пользователей;

- **Контроль контекста филиала (Salon Scoping)** – функция `resolve_salon_scope(current_user, salon_id)` осуществляет явное разграничение области видимости данных внутри защищённых эндпоинтов. Для роли `owner` отсутствие параметра `salon_id` означает доступ ко всей сети филиалов, тогда как для роли `admin` доступ строго ограничен закреплённым за ним филиалом (при попытке несанкционированного доступа возвращается HTTP-код 403);

- **Единый механизм выборки списков** – фильтрация, сортировка и постраничная выдача не дублируются в каждом репозитории, а вынесены в общие средства (`repositories/_query_utils.py`, `schemas/pagination.py`). Параметры страницы объявлены переиспользуемой зависимостью `PageParams` с проверкой границ на уровне схемы (`page ≥ 1`, `page_size` – от 1 до 20, по умолчанию 20), а ответ обернут обобщённой моделью `PageResponse`, которая, помимо самих элементов, возвращает общее число строк и вычисляемое количество страниц. Функция `paginated()` выполняет уже отфильтрованный и отсортированный запрос дважды – подсчёт `COUNT` по подзапросу без `ORDER BY` и выборку страницы через `LIMIT/OFFSET`. Каталог мастеров (`MasterRepository.list_paginated`) поддерживает одновременную фильтрацию по четырём критериям (подстрока специализации, оказываемая услуга, точка сети, признак активности) и сортировку по имени либо по итоговой цене выбранной услуги в обоих направлениях. Пользовательский ввод, попадающий в оператор `ILIKE`, проходит через функцию `escape_like()`, экранирующую символы шаблона `%` и `_`: без этого запрос по строке «100 %» искажал бы выдачу и провоцировал дорогие полные просмотры таблицы.

2.3 Архитектура фронтенда

Клиентская часть приложения (`frontend/src/`) построена на базе фреймворка Vue 3 с применением Composition API (с повсеместным использованием синтаксиса `<script setup>` на базе JavaScript), маршрутизатора Vue Router 4 и библиотеки управления состоянием Pinia. Централизованное состояние разделено на шесть изолированных хранилищ (Setup Stores): `auth`, `salon`, `setup`, `siteContent`, `masterProfile` и `toast`.

Маршрутизация реализована в модуле `router/index.js` с применением

динамического импорта (ленивой загрузки) компонентов страниц. Глобальный навигационный хук `beforeEach` выполняет ролевую модель разграничения доступа на основе метаполей маршрутов (`meta.roles`), а также перенаправляет пользователей на интерфейс мастера первоначальной настройки системы (`/setup`) до завершения базовой конфигурации сервиса (5.3).

В архитектуре фронтенда реализованы два ключевых решения, обеспечивающих безопасность и гибкость клиентской части:

- **Безопасное хранение и обновление токенов аутентификации** – токен сессии передаётся исключительно через cookie-файлы с флагом `HttpOnly`, исключая его хранение в `localStorage` или переменных JavaScript, что минимизирует риски компрометации через XSS-атаки. Обновление пары токенов реализовано через перехватчик (`interceptor`) библиотеки `Axios` по паттерну `Single-Flight`: при получении ответа с кодом 401 инициируется единственный запрос к эндпоинту `POST/auth/refresh`, в то время как параллельные асинхронные вызовы ожидают разрешения единого промиса (`Promise`). Полный сброс сессии и перенаправление на форму входа (`logout`) выполняются только в случае неуспешного ответа самого запроса на обновление токена.
- **Динамическая тема оформления** – корпоративная цветовая палитра (`brand, accent, ink, stone`) объявлена в конфигурации `Tailwind CSS` через пользовательские CSS-переменные (`CSS Custom Properties`). Значения переменных динамически считываются из конфигурации сайта в `runtime`, что позволяет адаптировать визуальный стиль под бренд конкретного филиала без повторной компиляции исходного кода фронтенда.

Структуру интерфейса и его поведение при нештатных состояниях определяют следующие решения:

- **Навигация** – публичная часть сайта использует горизонтальное главное меню, тогда как оба закрытых раздела построены на боковой панели навигации (компонент `components/SidebarLink.vue`): административная панель содержит восемь разделов (статистика, пользователи, услуги, мастера, точки сети, отзывы, отчёты, настройки), личный кабинет мастера – три (записи, расписание, отзывы). Состав панели зависит от роли: пункты «Точки сети» и «Настройки» отображаются только владельцу сети, поскольку соответствующие эндпоинты закрыты для роли `admin` (2.2).
- **Адаптивная вёрстка** – разметка построена по принципу «сна-

чала мобильные» (mobile first): базовые классы описывают отображение на экране шириной от 320 пикселей, а перестроение макета выполняют два порога Tailwind CSS – sm (от 640 пикселей, планшет) и lg (от 1024 пикселей, десктоп). Документ объявляет viewport с width=device-width, вследствие чего мобильный браузер не масштабирует страницу целиком. Через эти пороги переключаются все элементы, для которых единый макет невозможен: в публичной части главное меню отображается строкой в шапке от 640 пикселей и переносится в отдельную полосу под шапкой с горизонтальной прокруткой на меньших экранах; в закрытых разделах боковая панель постоянно видима от 1024 пикселей и на меньших экранах заменяется выдвижной панелью, вызываемой кнопкой в шапке; сетки карточек (мастера, услуги, показатели панели администратора) меняют число колонок с одной на две и далее до пяти. Широкие табличные отчёты, которые нельзя осмысленно сузить, помещены в контейнер с собственной горизонтальной прокруткой, поэтому горизонтальная прокрутка страницы целиком не возникает ни на одном из трёх типов устройств.

- **Двусторонняя валидация форм** – ограничения полей проверяются дважды. На клиенте набор функций-валидаторов (utils/validators.js) в связке с composable-функцией composables/useFormErrors.js подсвечивает ошибку под полем до отправки запроса; на сервере те же ограничения независимо проверяются схемами Pydantic. Клиентская проверка дублирует серверные правила и не может их ослаблять: единственным авторитетом остаётся сервер, а клиентская половина лишь заменяет ответ с кодом 422 немедленной подсветкой поля.

- **Система интерфейсных компонентов** – визуальный стиль задан не набором разрозненных классов, а библиотекой из двадцати базовых компонентов (components/ui/: элементы ввода, кнопки, карточки, диалоги подтверждения, индикаторы состояния загрузки, уведомления). Оформление опирается на согласованные наборы значений, объявленные в конфигурации Tailwind CSS: палитра, шкала скруглений и трёхступенчатая шкала теней. Такое разделение позволяет менять оформление централизованно, не затрагивая разметку отдельных страниц.

- **Страницы ошибочных состояний** – обращение к несуществующему маршруту отображает страницу с кодом 404 и переходом в каталог

мастеров. Ответ сервера с кодом 5xx перехватывается тем же перехватчиком Axios, что и код 401, и приводит к переходу на страницу с кодом 500, содержащую пояснение о характере сбоя и контактный телефон точки сети. Сетевые ошибки (отсутствие ответа сервера) к такому переходу намеренно не приводят: их причина, как правило, находится на стороне клиента, и переход на страницу ошибки означал бы потерю заполненной формы.

– **Автоматическое обновление статистики** – показатели административной панели обновляются без участия пользователя с интервалом 30 секунд (composables/usePolling.js). Очередной запрос планируется только после завершения предыдущего, что исключает наложение запросов при медленном ответе сервера; на неактивной вкладке браузера опрос приостанавливается и возобновляется немедленным обновлением при возврате к ней. Сбой фоновое обновления не прерывает работу с панелью: ранее полученные значения сохраняются на экране с отметкой времени их получения.

2.4 Требования законодательства и защита данных

В соответствии с техническим заданием (подраздел 1.4), система спроектирована с учётом требований Федерального закона № 152-ФЗ «О персональных данных» [8] и Федерального закона № 149-ФЗ «Об информации, информационных технологиях и о защите информации» [9]. Методология оценки достаточности реализованных мер защиты опирается на положения ГОСТ Р ИСО/МЭК 15408-3-2008 [10].

В рамках программного решения реализован следующий комплекс мер по обеспечению информационной безопасности и защите персональных данных:

– **Фиксация согласия на обработку персональных данных** – форма регистрации предусматривает обязательное подтверждение согласия пользователя посредством интерактивного элемента (чекбокса) с активной гиперссылкой на текст политики конфиденциальности. Отправка формы клиентом блокируется до момента подтверждения; аналогичный механизм валидации согласия реализован для сценария авторизации через внешний сервис VK ID.

– **Публикация правовых документов** – на клиентском маршруте

/privacy размещена страница политики конфиденциальности, структурированная в соответствии с требованиями 152-ФЗ (определены правовые основания, категории обрабатываемых данных, цели обработки, права субъектов, условия применения аналитических инструментов и контактные данные оператора).

- **Информирование об использовании cookie-файлов и сервисов аналитики** – в п. 8 политики конфиденциальности раскрыты сведения об обработке cookie-файлов и интеграции со счётчиком Яндекс.Метрики (5.5).

- **Безопасность хранения и управления данными** – аутентификационные данные не сохраняются в открытом виде: в базе данных фиксируются исключительно односторонние криптографические хеши (поле password_hash). Физическое удаление учётной записи пользователя при наличии связанных с ней записей на приём предотвращается на уровне схемы реляционной БД с помощью ограничения ссылочной целостности (ON DELETE RESTRICT). Для прекращения обслуживания учётной записи применяется механизм логической блокировки (флаг is_blocked).

- **Вход через внешнего поставщика учётных записей** – помимо пары «адрес электронной почты и пароль», поддерживается авторизация через сервис VK ID по протоколу OAuth 2.0. Применён классический серверный поток с перенаправлением браузера (GET/auth/vk/login → VK → GET/auth/vk/callback), дополненный механизмом PKCE (метод S256) и параметром state; оба значения хранятся в cookie-файлах с флагом HttpOnly, ограниченных путём маршрута обратного вызова, и защищают обмен кода на токен от подмены и от межсайтовой подделки запроса. Учётная запись, созданная таким способом, не содержит пароля вовсе: поле password_hash допускает значение NULL, а связь с внешней учётной записью хранится в поле vk_user_id. Для ролей owner и admin этот способ входа отключается, если задан код администратора: в потоке OAuth предъявить его негде, и такой вход обходил бы второй фактор.

- **Противодействие подбору учётных данных** – каждая попытка входа фиксируется в таблице login_attempt_log с указанием адреса электронной почты, IP-адреса и результата. После пяти неудачных попыток вход по данному адресу временно блокируется на 15 минут; запросы на восстановление пароля ограничены тремя в час. Журнал попыток одновременно

выполняет функцию аудита: он позволяет отличить единичную ошибку пользователя от целенаправленного перебора.

- **Второй фактор доступа к административной панели** – помимо пароля, роли owner и admin предъявляют при входе общий код администратора (параметр конфигурации ADMIN_LOGIN_TOKEN). Код проверяется после пароля и сравнивается функцией secrets.compare_digest, устойчивой к атакам по времени выполнения; неудачная проверка фиксируется в том же журнале login_attempt_log, вследствие чего подбор кода ограничен той же блокировкой, что и подбор пароля. При неверном пароле сообщение об ошибке не изменяется даже при верном коде – в противном случае различие в ответах указывало бы на успешный подбор пароля. Для ролей client и master код не запрашивается, поскольку административная панель закрыта для них ролевой моделью (2.2). Параметр является необязательным: при пустом значении вход выполняется по паролю, что позволяет разворачивать систему без дополнительной настройки.

- **Защита от межсайтовых запросов** – cookie-файлы с токенами выпускаются с атрибутом SameSite=Lax, вследствие чего браузер не передаёт их при межсайтовых запросах методами POST, PATCH и DELETE. Перечень источников, которым разрешено обращаться к программному интерфейсу, задаётся явным списком в конфигурации сервера (механизм CORS) и не допускает произвольного происхождения запроса.

- **Заголовки безопасности ответа** – обратный прокси-сервер Caddy [13] служит единственной точкой входа и перед клиентской, и перед серверной частью, поэтому набор защитных заголовков задан централизованно в одном месте (Caddyfile) и покрывает оба приложения сразу:

- Content-Security-Policy с директивой default-src'self' – источники сценариев, стилей, шрифтов и соединений перечислены явно; директива script-src обходится без послабления 'unsafe-inline', ради чего счётчик Яндекс.Метрики и вынесен в отдельный статический файл (5.5). Директивы frame-ancestors'none', form-action'self', base-uri'self' и object-src'none' закрывают встраивание страницы, отправку форм на посторонний адрес и подмену базового адреса документа;

- Strict-Transport-Security со сроком в один год и распространением на поддомены – браузер обращается к сервису исключительно по HTTPS,

не выполняя первичный запрос по незащищённому протоколу;

- X-Content-Type-Options: nosniff, X-Frame-Options: DENY и Referrer-Policy: strict-origin-when-cross-origin – запрет угадывания типа содержимого, защита от кликджекинга и ограничение утечки адреса страницы во внешние переходы;

- Permissions-Policy – геолокация, микрофон и камера отключены для страницы полностью, поскольку не используются ни одним сценарием;

- заголовок Server удаляется из ответа (директива -Server), чтобы не раскрывать используемое серверное программное обеспечение и его версию.

3 Обоснование выбора средств реализации

3.1 Метод экспертных оценок

Для обоснованного выбора технологического стека применяется метод ранжировки с расчётом интегрального показателя полезности. Метод позволяет формализовать экспертную оценку при сравнении альтернатив по нескольким критериям с различным приоритетом; экспертом выступает единственный исполнитель проекта.

Коэффициент ценности C_{ij} для произвольного ранга критерия r_{ij} ($1 \leq r_{ij} \leq n$):

$$C_{ij} = 1 - \frac{r_{ij} - 1}{n}, \quad (1)$$

где C_{ij} – коэффициент ценности i -го критерия;
 r_{ij} – ранг критерия (1,0 – наиболее значимый);
 n – общее число критериев.

Нормированный вес i -го критерия b_i :

$$b_i = C_{ij} / \sum_{i=1}^n C_{ij}. \quad (2)$$

Интегральный показатель полезности альтернативы E :

$$E = \sum_{i=1}^n b_i q_i, \quad (3)$$

где E – интегральный показатель полезности альтернативы;
 b_i – нормированный вес i -го критерия;
 $q_i \in [0; 1]$ – нормированная оценка альтернативы по i -му критерию (1,00 присваивается лучшей альтернативе по данному критерию).

3.2 Список альтернатив

Рассматриваются технологии с открытой лицензией и широкой распространённостью на российском рынке разработки; учитывается доступность специалистов и отсутствие лицензионных ограничений для разработ-

чиков в РФ.

Серверная часть. Рассматриваются Python, Node.js, Java и PHP. Python обладает развитой экосистемой библиотек для работы с базами данных, построения REST API и организации фоновых задач. Node.js – популярная платформа для серверной разработки на JavaScript с активным сообществом. Java – зрелое корпоративное решение с богатой инфраструктурой, но с более высоким порогом входа и длительной первоначальной настройкой. PHP сохраняет значительную долю рынка корпоративных веб-приложений. При выборе серверной части приоритет отдан доступности специалистов и скорости разработки: нагрузка малой парикмахерской или сети из нескольких точек не является критичным фактором, поэтому производительность и масштабируемость имеют меньший вес. Критерии оценивания серверной части приведены в соответствии с таблицей 1.

Таблица 1 – Критерии и веса оценки серверной части

Обозначение	Критерий	Ранг r	C_i	b_i
q_1	Производительность	6,5	0,313	0,070
q_2	Надёжность, наличие готовых библиотек	4	0,625	0,139
q_3	Стоимость и лицензирование	3	0,750	0,167
q_4	Безопасность	2	0,875	0,194
q_5	Документация и сообщество	8	0,125	0,028
q_6	Масштабируемость	6,5	0,313	0,070
q_7	Скорость разработки	1	1,000	0,222
q_8	Простота интеграции со сторонними сервисами	5	0,500	0,111

Клиентская часть. Рассматриваются Vue 3, React, Next.js и Angular. Vue 3 и React – наиболее распространённые библиотеки для построения одностраничных приложений (SPA). Next.js – фреймворк на основе React с поддержкой серверного рендеринга (SSR), избыточного для проекта без требований к SEO публичной части. Angular – полноценный фреймворк со

встроенной маршрутизацией, управлением состоянием и внедрением зависимостей, но с более высоким порогом входа. При выборе клиентской части приоритет отдан скорости разработки UI и низкому порогу входа: система включает как публичный интерфейс записи, так и административную панель, разрабатываемые одним исполнителем. Критерии оценивания клиентской части приведены в соответствии с таблицей 2.

Таблица 2 – Критерии и веса оценки клиентской части

Обозначение	Критерий	Ранг r	C_i	b_i
q_1	Скорость разработки UI, компактность API реактивности	1	1,000	0,400
q_2	Порог входа для стороннего разработчика	2	0,750	0,300
q_3	Сложность интеграции с backend (REST API)	3	0,500	0,200
q_4	Качество документации	4	0,250	0,100

Хранение данных. Рассматриваются PostgreSQL, MariaDB и SQLite. Все три СУБД распространяются по открытым лицензиям. PostgreSQL – объектно-реляционная СУБД с развитыми механизмами обеспечения целостности данных и конкурентного доступа. MariaDB – развитие MySQL, широко применяемое в веб-разработке. SQLite – встраиваемая СУБД без отдельного серверного процесса, с существенными ограничениями в части конкурентной записи. При выборе СУБД целостность данных и конкурентный доступ – первостепенный критерий: несколько администраторов и клиентов одновременно работают с расписанием, и двойное бронирование одного временного слота должно быть исключено средствами самой СУБД (многоверсионное управление конкурентным доступом, MVCC; уровни изоляции транзакций; ограничения целостности), а не только проверкой в коде приложения. Критерии оценивания СУБД приведены в соответствии с таблицей 3.

Таблица 3 – Критерии и веса оценки СУБД

Обозначение	Критерий	Ранг r	C_i	b_i
q_1	Поддержка конкурентного доступа и изоляции транзакций	1	1,000	0,279
q_2	Ограничения целостности на уровне схемы (EXCLUDE, диапазонные типы)	2	0,833	0,233
q_3	Простота администрирования и резервного копирования	3	0,667	0,186
q_4	Поддержка со стороны российских managed-хостингов	4	0,500	0,140
q_5	Производительность при индексируемых выборках	5	0,333	0,093
q_6	Лицензия и отсутствие санкционных ограничений	6	0,167	0,047

3.3 Итоговые средства реализации

Итоговые результаты экспертной оценки представлены в соответствии с таблицей 4.

Таблица 4 – Итоговые результаты экспертной оценки

Категория	Средство	E
Серверная часть	Python (FastAPI)	0,904
Клиентская часть	Vue 3	0,930
СУБД	PostgreSQL	0,978

Python демонстрирует наивысший интегральный показатель (0,904) прежде всего за счёт лидерства по критерию скорости разработки, имеющему

наибольший вес (0,222). Фреймворк FastAPI [1] обеспечивает встроенную валидацию данных через Pydantic [7], автоматическую генерацию интерактивной документации API (OpenAPI/Swagger, /api/docs) и асинхронную обработку запросов – полноценный REST API реализуется за минимальное время без отдельного описания контракта вручную. Экосистема Python предоставляет зрелые библиотеки для всех задач проекта: SQLAlchemy 2.0 [2] (типизированный ORM), Alembic [11] (версионизируемые миграции схемы), python-jose (JWT). Весь стек распространяется под открытыми лицензиями без ограничений для российских разработчиков.

Фреймворк Vue 3 занял лидирующую позицию с интегральным показателем 0,930 благодаря максимальной оценке по критерию скорости разработки пользовательского интерфейса (весовой коэффициент 0,400). Использование Composition API в сочетании с библиотекой управления состоянием Pinia позволяет создавать реактивный интерфейс без избыточного шаблонного кода (boilerplate), характерного для связок Vuex/Redux или системы внедрения зависимостей Angular, что оптимально для масштаба разрабатываемого сервиса. Интеграция с REST API бэкенда стандартизирована, а исчерпывающая документация фреймворка минимизирует порог входа и упрощает сопровождение клиентской части.

PostgreSQL демонстрирует наивысший интегральный показатель (0,978) и уверенно лидирует по двум наиболее весомым критериям. По конкурентному доступу PostgreSQL не имеет равных среди рассмотренных альтернатив: механизм MVCC и поддержка уровней изоляции транзакций гарантируют корректность при одновременных попытках бронирования одного слота несколькими клиентами. Дополнительную защиту обеспечивает ограничение целостности EXCLUDE USING gist с диапазоным типом tstzrange: требование «0 двойных бронирований» (техническое задание, подраздел 2.2) решено на уровне схемы БД, а не проверкой в коде приложения перед записью, которая при параллельных запросах подвержена состоянию гонки. PostgreSQL распространяется под лицензией PostgreSQL License (аналог MIT) и доступен в виде managed-сервиса на отечественных облачных платформах. SQLite исключён ввиду отсутствия полноценной поддержки конкурентной записи; MariaDB уступает PostgreSQL по глубине поддержки механизмов изоляции

транзакций и диапазонных ограничений целостности.

Вспомогательные средства реализации, не участвовавшие в сравнении альтернатив ввиду отсутствия конкурирующих решений того же назначения в рамках проекта, – Tailwind CSS (единая цветовая и типографическая система без написания собственного CSS-фреймворка с нуля) и Docker Compose [12] с Caddy [13] (воспроизводимое развёртывание и автоматический TLS-сертификат Let’s Encrypt без ручной настройки nginx/certbot; подробности – руководство администратора, раздел 1).

Обоснование отказа от распределённой очереди задач (Celery / Redis). Рассылка уведомлений клиентам реализована в виде фонового асинхронного цикла (asynсio) внутри основного процесса бэкенда без привлечения внешнего брокера сообщений (техническое задание, подраздел 3.3). Архитектура системы рассчитана на эксплуатацию в рамках одного контейнера backend (руководство администратора, раздел 1). В этих условиях внедрение внешнего брокера очередей привело бы к неоправданному усложнению инфраструктуры и увеличению накладных расходов. По аналогичной причине ограничение частоты запросов (rate limiting) и защита от подбора аутентификационных данных реализованы без использования внешнего кэша Redis: лимиты запросов контролируются библиотекой slowapi в оперативной памяти процесса по IP-адресам, а фиксация попыток входа осуществляется в служебной таблице login_attempt_log базы данных.

3.4 Дополнительные инструменты разработки

Вспомогательные инструменты линтинга, тестирования и непрерывной интеграции, использованные при разработке, перечислены в соответствии с таблицей 5.

Таблица 5 – Инструменты разработки, тестирования и контроля качества

Инструмент	Назначение
ruff	Линтер Python (E,F,B,UP,SIM,C4).
ESLint 9 + eslint-plugin-vue	Линтер JS/Vue (flat config, vue/flat/essential).

Продолжение таблицы 5

Инструмент	Назначение
Pyright	Проверка типов Python в режиме basic (локально).
pytest (unit + integration)	Модульные тесты (backend/tests) и отдельно помеченные интеграционные тесты (backend/tests/integration) на настоящем Postgres через httpx.AsyncClient – покрывают ограничение EXCLUDE, уникальность slug точки, область видимости по точкам сети, полный сценарий записи «от начала до конца».
Vitest + @vue/test-utils	Модульные тесты фронтенда (хранилища, composables).
Playwright + @axe-core/playwright	End-to-end тесты в браузере (Chromium) и автоматическая регрессия по доступности (accessibility).
GitHub Actions (CI)	Параллельные job'ы на каждый push/PR в main: тесты и линт бэкенда, интеграционные тесты бэкенда на реальном Postgres, сборка и линт фронтенда, unit- и e2e-тесты фронтенда, синтаксическая проверка docker-compose.yml.
pip-audit / npm audit	Проверка зависимостей на известные уязвимости как отдельные шаги CI.
Dependabot	Еженедельные автоматические PR на обновление зависимостей (pip, npm, Docker-образы обоих Dockerfile, GitHub Actions).

Работа велась в системе контроля версий Git с размещением репозитория на сервисе GitHub. Принята двухветочная модель: ветвь main содержит состояние, соответствующее развёрнутой версии, разработка ведётся в ветви develop с последующим слиянием. К моменту подготовки настоящего

документа история насчитывает более 150 фиксаций (commit); сообщение каждой из них оформлено по соглашению Conventional Commits (префиксы feat, fix, docs, chore) и содержит пояснение причины изменения, а не только его состава.

4 Структура базы данных

4.1 Инфологическая модель

Модель разделена на две схемы для читаемости: в соответствии с рисунком 2 – сущности бизнес-домена (точки, пользователи, мастера, услуги, записи, отзывы); в соответствии с рисунком 3 – сущности, обеспечивающие безопасность входа (сессии, токены, попытки входа), и настройки сайта.

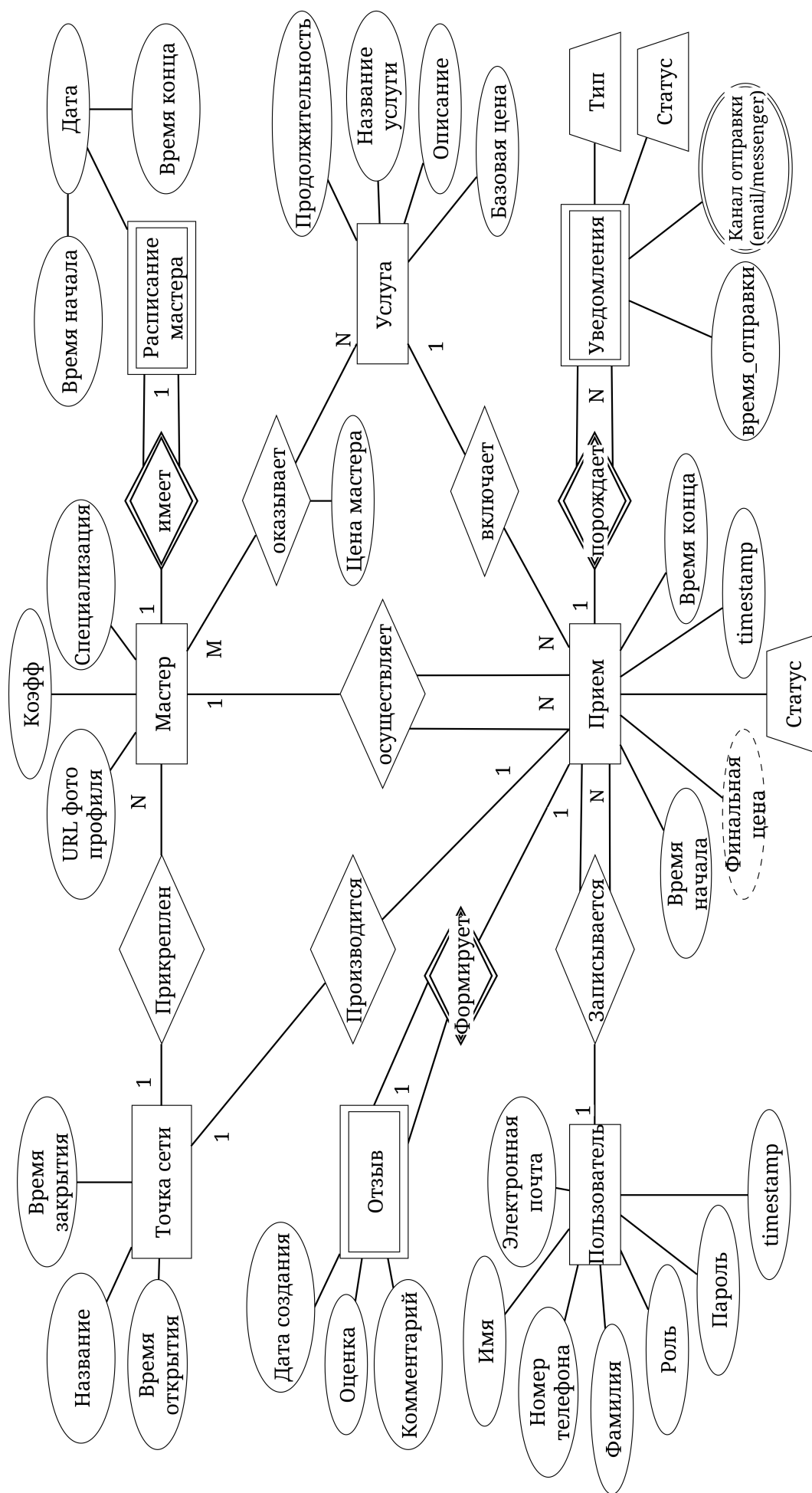


Рисунок 2 – Инфологическая модель – бизнес-домен (актуальная схема БД)

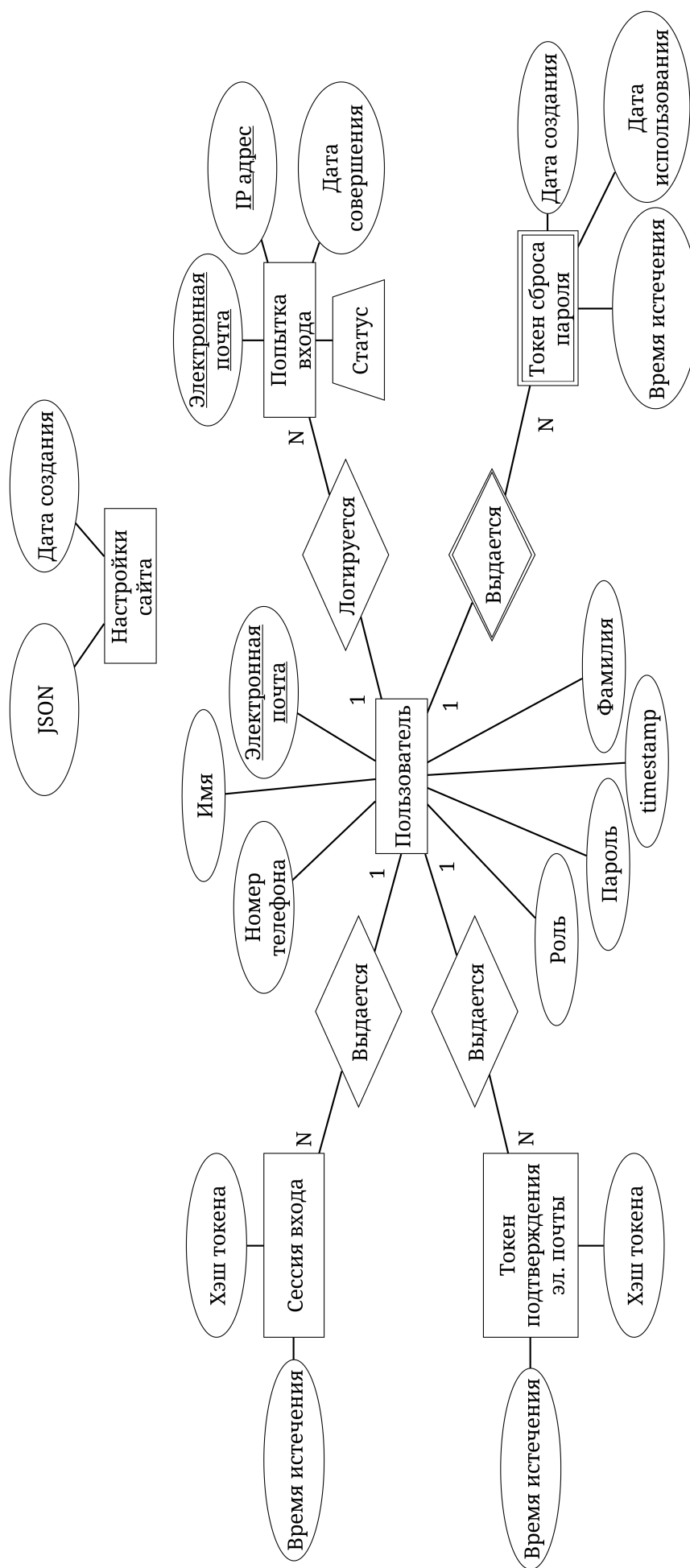


Рисунок 3 – Инфологическая модель – аутентификация и настройки сайта

4.2 Даталогическая модель

Именованние объектов схемы подчинено принятому в проекте стандарту: идентификаторы записаны в нотации snake_case строчными латинскими буквами, имя таблицы задаётся в единственном числе и отражает тип сущности, а зарезервированные слова SQL не используются – отсюда user_account вместо user. Связующая таблица именуется по двум связываемым сущностям в алфавитном порядке (master_service), таблица журнала – с суффиксом _log (login_attempt_log). Поля внешних ключей следуют шаблону «имя целевой таблицы + _id»; при нескольких связях с одной и той же таблицей добавляется префикс роли (client_id и master_id в таблице appointment – обе ссылаются на user_account).

Фактическая схема БД насчитывает 13 таблиц (15 миграций Alembic [11] в директории backend/alembic/versions/):

- 1) salon – точки сети (адрес, часы работы, slug);
- 2) user_account – учётные записи всех ролей (client/master/admin/owner);
- 3) master – профиль мастера (специализация, коэффициент к цене);
- 4) schedule – рабочее расписание мастера по дням недели;
- 5) service – каталог услуг (цена, длительность);
- 6) master_service – прайс услуги, индивидуальный по мастеру;
- 7) appointment – записи на приём (EXCLUDE USING gist – защита от двойного бронирования на уровне СУБД);
- 8) review – отзывы клиентов о завершённых записях;
- 9) user_session – серверное хранилище refresh-сессий;
- 10) password_reset_token – одноразовые токены сброса пароля;
- 11) email_verification_token – одноразовые токены подтверждения email;
- 12) login_attempt_log – аудит попыток входа;
- 13) site_setting – редактируемый контент сайта (единственная строка, JSONB).

Схема приведена к третьей нормальной форме. Помимо базовых типов применяются расширенные типы данных СУБД PostgreSQL. Перечис-

лимые типы (ENUM) объявлены на уровне схемы, а не эмулируются строковыми полями с проверочным ограничением: тип `user_role` задаёт четыре роли (`client`, `master`, `admin`, `owner`), тип `appointment_status` – четыре состояния записи (`pending`, `confirmed`, `cancelled`, `done`), образующие машину состояний записи на приём. Тип JSONB использован для таблицы `site_setting`, хранящей редактируемый администратором контент сайта переменной структуры.

Удаление данных реализовано в двух формах, различающихся по назначению. Для учётных записей, профилей мастеров и услуг применяется логическое (мягкое) удаление: примесный класс `SoftDeleteMixin` добавляет таблице поле `deleted_at`, а сама строка сохраняется, вследствие чего связанные с ней записи на приём и отзывы остаются доступными для отчётности. Уникальность значений при этом обеспечивают частичные индексы с условием `WHERE deleted_at IS NULL` (4.3), что позволяет повторно зарегистрировать адрес электронной почты, ранее принадлежавший удалённой учётной записи. Для точек сети мягкое удаление намеренно не применяется: точка либо закрывается флагом `is_active`, либо не удаляется вовсе – физическое удаление предотвращается ограничением ссылочной целостности `ON DELETE RESTRICT` со стороны мастеров и записей.

Поведение внешних ключей при удалении родительской строки задано явно для каждой связи, а не оставлено значением по умолчанию, и выбор действия подчинён одному правилу: строка, не имеющая смысла в отрыве от родителя, удаляется вместе с ним, а строка, представляющая хозяйственный факт, удалению препятствует.

- `ON DELETE CASCADE` назначено связям с подчинёнными данными, время жизни которых полностью определяется владельцем: профиль мастера и его сессии, одноразовые токены сброса пароля и подтверждения адреса электронной почты удаляются вместе с учётной записью пользователя; строки связующей таблицы `master_service` – вместе с мастером или услугой; отзывы – вместе с записью на приём, к которой они относятся. Тем самым осиротевшие строки в схеме невозможны, а удаление учётной записи по требованию субъекта персональных данных (2.4) не требует ручной зачистки связанных таблиц.

- `ON DELETE RESTRICT` назначено связям, ведущим к записям на приём: клиент, мастер, услуга и точка сети, на которые ссылается хотя

бы одна запись, физически удалены быть не могут. Именно это ограничение и делает необходимым описанное выше мягкое удаление: попытка удалить строку возвращается ошибкой СУБД, поэтому единственным доступным приложению способом остаётся проставление `deleted_at`, при котором история приёмов и отчётность за прошедшие периоды сохраняются в неизменном виде.

– `ON DELETE SET NULL` применено в единственном месте – в журнале попыток входа `login_attempt_log`: сама запись журнала переживает удаление учётной записи, теряя лишь ссылку на неё, поскольку значение журнала как средства аудита не зависит от того, существует ли пользователь на текущий момент.

Даталогическая модель данных приведена в соответствии с рисунком 4.

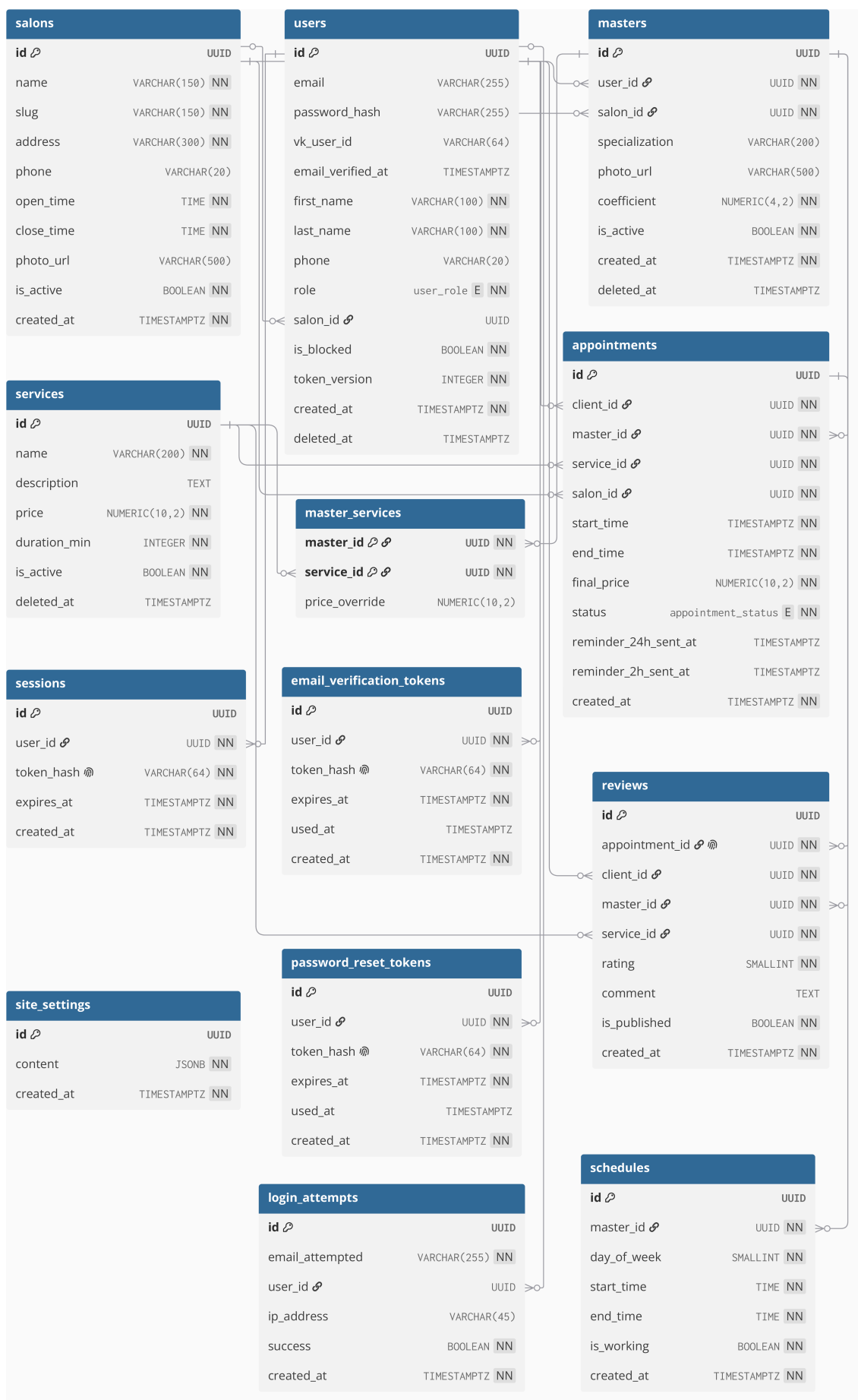


Рисунок 4 – Даталогическая модель – физическая схема БД

4.3 Индексы и оптимизация выборки

Помимо индексов, создаваемых СУБД автоматически для первичных ключей и ограничений уникальности, схема содержит 21 явно объявленный индекс. Индексы создаются миграциями Alembic вместе с таблицами, к которым относятся, и объявлены в моделях предметной области (backend/app/models/), вследствие чего схема, описанная в коде, и схема, существующая в базе данных, не расходятся между собой. Имена индексов подчинены тому же стандарту, что и имена таблиц (4.2): обычный индекс получает префикс idx_, уникальный – uniq_, далее следуют имя таблицы и перечень полей в порядке их объявления. Назначение основных индексов приведено в соответствии с таблицей 6.

Таблица 6 – Индексы схемы данных и их назначение

Индекс	Назначение
uniq_user_account_email	Частичный уникальный индекс по адресу электронной почты с условием deleted_at IS NULL. Обеспечивает уникальность адреса среди действующих учётных записей и одновременно обслуживает поиск пользователя при входе. Аналогично построены индексы по номеру телефона и идентификатору VK ID.
idx_user_account_role	Выборка пользователей по роли – список клиентов и назначение прав в административной панели.
idx_user_account_salon_id	Частичный индекс по точке сети с условием salon_id IS NOT NULL: разграничение области видимости данных для роли admin (2.2).

Продолжение таблицы 6

Индекс	Назначение
idx_appointment_start_time_end_time	Составной индекс по границам интервала (start_time, end_time) – расчёт свободных интервалов мастера, наиболее частая выборка в системе.
idx_appointment_master_id	Записи конкретного мастера – личный кабинет и расписание.
idx_appointment_client_id	Записи конкретного клиента – история посещений в личном кабинете.
idx_appointment_status	Отбор записей по состоянию: рассылка напоминаний и отчёты по завершённым записям.
idx_appointment_salon_id	Записи в границах одной точки сети – статистика и отчёты филиала.
idx_master_salon_id	Мастера, закреплённые за точкой сети.
idx_service_name	Поиск услуги по названию в каталоге.
idx_review_master_id	Отзывы о мастере и расчёт его средней оценки.
idx_review_service_id	Отзывы об услуге.
idx_login_attempt_log_email_attempted_created_at	Составной индекс (email_attempted, created_at): подсчёт неудачных попыток входа за последние 15 минут (2.4). Выполняется при каждой попытке входа, вследствие чего отсутствие индекса замедляло бы авторизацию по мере роста журнала.
idx_user_session_user_id	Активные сессии пользователя: обновление токена и отзыв всех сессий администратором.
idx_password_reset_token_user_id	Одноразовые токены сброса пароля по владельцу.

Продолжение таблицы 6

Индекс	Назначение
idx_email_verification_token_user_id	Одноразовые токены подтверждения адреса электронной почты по владельцу.
uniq_salon_slug	Уникальность и поиск точки сети по человекочитаемому идентификатору в адресе страницы.
idx_master_service_service_id	Обратный обход связи «многие ко многим»: перечень мастеров, оказывающих заданную услугу.

Отдельно следует отметить ограничение EXCLUDE USING gist на таблице appointment (4.2), которое требует расширения btree_gist и опирается на собственный индекс типа GiST. Это ограничение выполняет двойную функцию: предотвращает пересечение интервалов времени у одного мастера на уровне СУБД и ускоряет проверку пересечений, поскольку поиск конфликтующего интервала выполняется по индексу, а не последовательным просмотром таблицы.

Корректность применения индексов проверяется командой EXPLAIN ANALYZE на плане запроса расчёта занятых интервалов мастера – наиболее частой выборки системы. Запрос отбирает время начала и окончания записей одного мастера за одни сутки, исключая отменённые; его условия видны в строках Recheck Cond и Filter самого плана. Замер выполнен на базе, наполненной 40 000 записей на приём за два года по восьми мастерам; перед замером собрана статистика (ANALYZE). План выполнения приведён в соответствии с рисунком 5.

QUERY PLAN

```
Bitmap Heap Scan on appointment (cost=64.08..147.85 rows=18
  width=16) (actual time=0.119..0.123 rows=24.00 loops=1)
  Recheck Cond: ((start_time >= '2025-06-01 00:00:00+03'::timestamp
    with time zone) AND (start_time < '2025-06-02
    00:00:00+03'::timestamp with time zone) AND (master_id =
    '31785464-6376-4956-9b25-52b88c074d84'::uuid))
  Filter: (status <> 'cancelled'::appointment_status)
  Heap Blocks: exact=4
  Buffers: shared hit=11
-> BitmapAnd (cost=64.08..64.08 rows=24 width=0) (actual
  time=0.112..0.112 rows=0.00 loops=1)
  Buffers: shared hit=7
  -> Bitmap Index Scan on
  idx_appointment_start_time_end_time (cost=0.00..6.21
  rows=192 width=0) (actual time=0.013..0.014 rows=192.00
  loops=1)
    Index Cond: ((start_time >= '2025-06-01
    00:00:00+03'::timestamp with time zone) AND
    (start_time < '2025-06-02 00:00:00+03'::timestamp
    with time zone))
    Index Searches: 1
    Buffers: shared hit=2
  -> Bitmap Index Scan on idx_appointment_master_id
  (cost=0.00..57.61 rows=4976 width=0) (actual
  time=0.097..0.097 rows=5000.00 loops=1)
    Index Cond: (master_id =
    '31785464-6376-4956-9b25-52b88c074d84'::uuid)
    Index Searches: 1
    Buffers: shared hit=5

Planning:
  Buffers: shared hit=192
Planning Time: 0.302 ms
Execution Time: 0.149 ms
(19 rows)
```

Рисунок 5 – План выполнения запроса расчёта занятых интервалов

Планировщик выбрал не последовательный просмотр таблицы, а пересечение двух битовых карт (BitmapAnd), построенных по обоим индексам: idx_appointment_start_time_end_time отбирает записи в границах суток, idx_appointment_master_id – записи конкретного мастера. Из 40 000 строк прочитано четыре страницы данных (Heap Blocks: exact=4), время выполнения – 0,149 мс при времени планирования 0,302 мс, то есть построение плана обходится дороже самого чтения. Условие по состоянию записи применяется

уже после выборки (Filter), и отдельный индекс по status для этого запроса не нужен: он существует ради других выборок (таблица 6).

5 Реализация и развёртывание приложения

5.1 Общие сведения

Полный перечень операций по каждой из четырёх ролей – руководство пользователя, подразделы 4.1–4.4. Здесь – сквозные технические сценарии двух ключевых процессов на уровне взаимодействия фронтенда, бэкенда и базы данных (5.2, 5.3) и экранные формы разработанного приложения (5.4); диаграммы последовательностей пяти ключевых технических сценариев, детализирующих часть процессов из подраздела 1.2, приведены в приложении А.

5.2 Создание записи (бронирование)

Технический сценарий создания записи на уровне взаимодействия фронтенда, бэкенда и базы данных показан в соответствии с рисунком 6.

Повторная проверка доступности слота на шаге 6 на уровне СУБД, а не только на уровне кода перед вставкой, – прямая реализация цели «0 инцидентов двойного бронирования» из технического задания (подраздел 2.2): при конкурентной попытке два запроса могут одновременно пройти проверку на шаге 1–4, но лишь один пройдёт INSERT на шаге 6, второй получит 409 от самой базы данных.

5.3 Первый запуск системы (мастер /setup)

Сценарий первого запуска, выполняемый мастером настройки /setup, представлен в соответствии с рисунком 7.

Маршрут /setup проверяется единым глобальным `router.beforeEach`-хуком фронтенда перед *каждой* навигацией (раздел 2), поэтому попасть в остальную часть приложения, минуя этот сценарий, нельзя; после успешного шага 6 статус настройки самоблокируется навсегда (руководство администратора, подраздел 3.4).

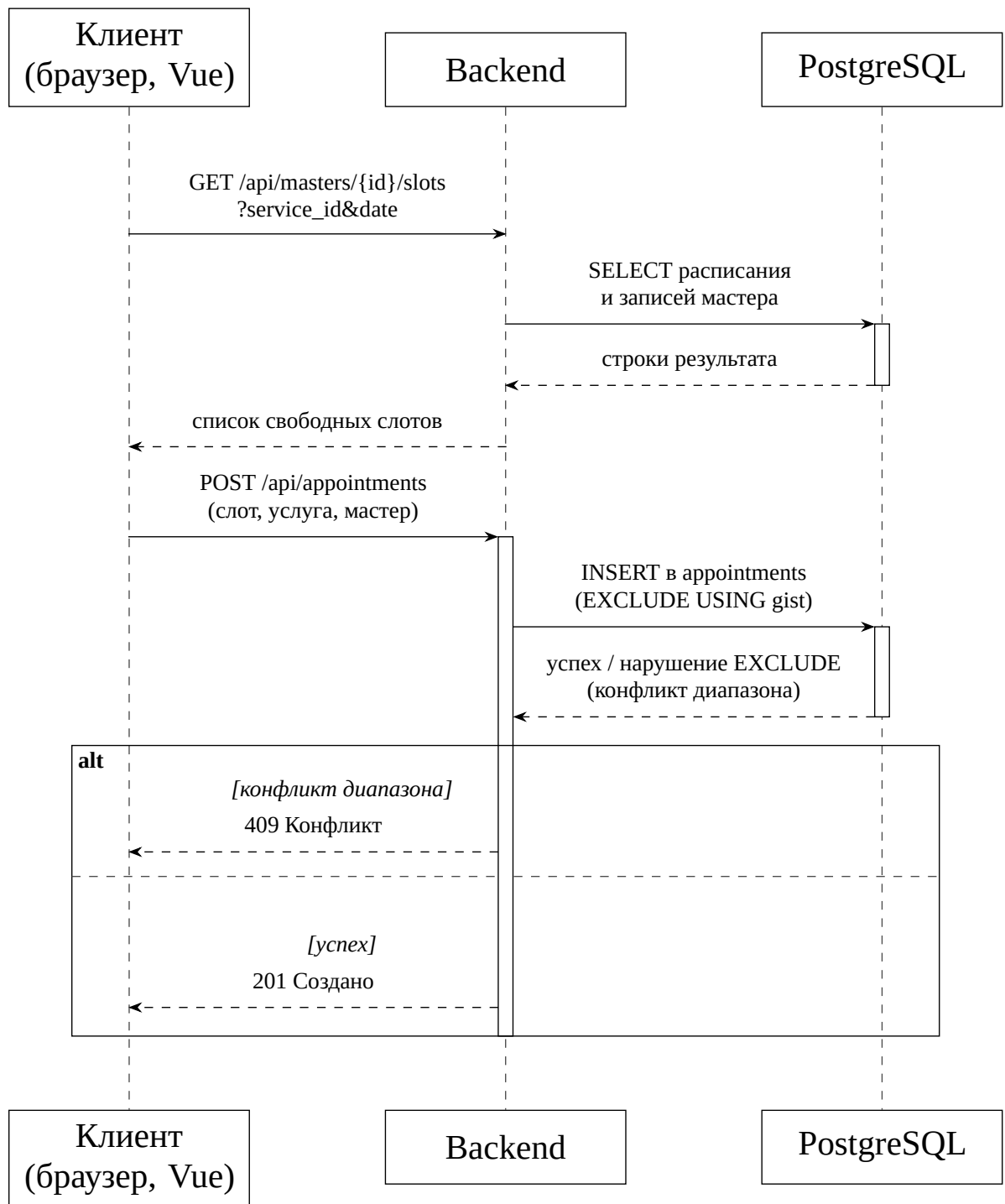


Рисунок 6 – Диаграмма последовательности: создание записи

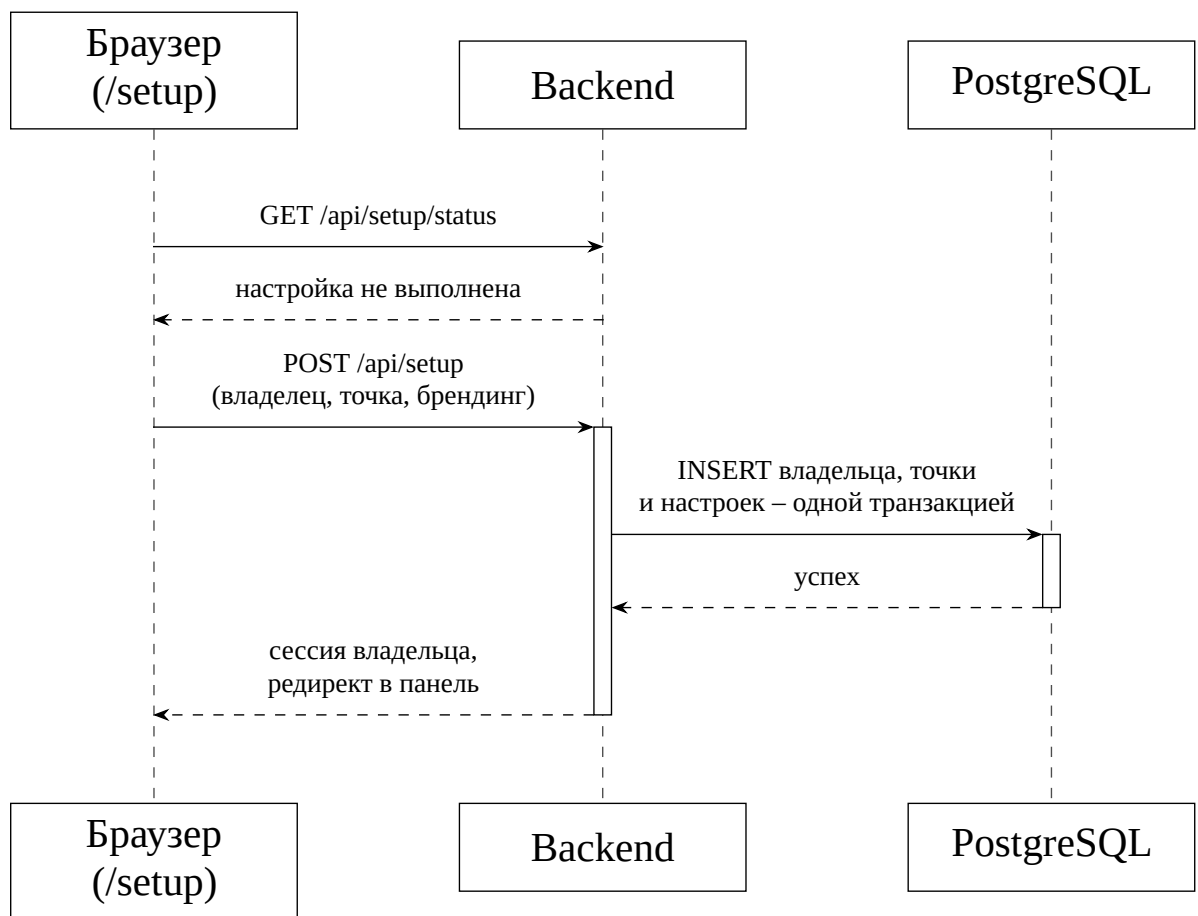


Рисунок 7 – Диаграмма последовательности: первый запуск (/setup)

5.4 Интерфейс разработанного приложения

Ниже приведены экранные формы, иллюстрирующие результат реализации архитектурных решений, описанных в разделе 2. Полное пошаговое описание всех экранов и элементов управления, включая формы регистрации, входа и восстановления пароля, вынесено в руководство пользователя; здесь показаны шесть форм, покрывающих обе части системы – публичную витрину и закрытые разделы.

Главная страница сайта в режиме редактирования приведена в соответствии с рисунком 8.

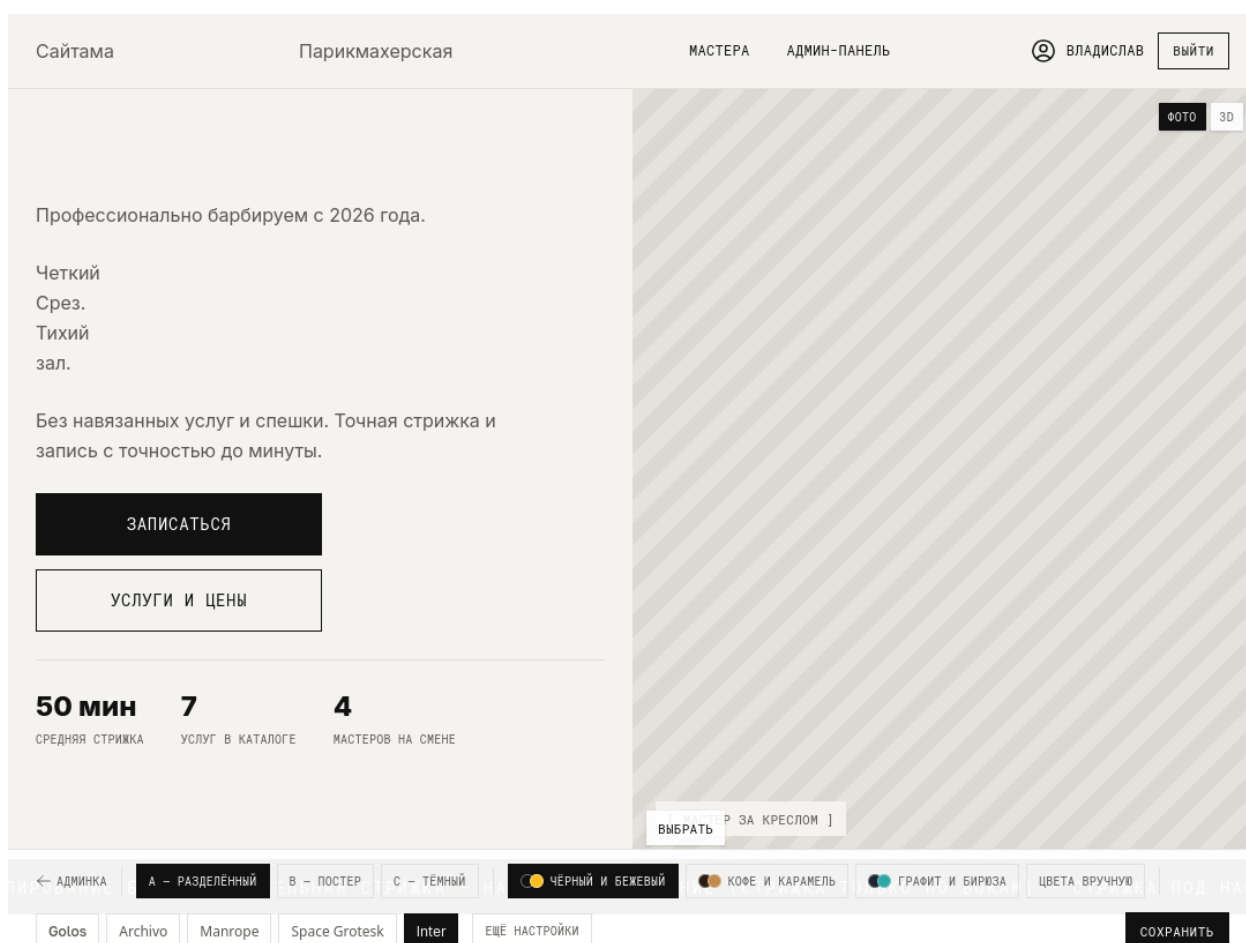


Рисунок 8 – Главная страница в режиме редактирования (/admin/settings)

Форма демонстрирует подход к управлению контентом: владелец сети редактирует тексты, изображение и оформление непосредственно на самой странице, а не в отдельной административной форме. Нижняя панель переключает готовые наборы оформления и шрифты; выбранные значения сохраняются в таблицу `site_setting` (4.2) и применяются фронтендом через

CSS-переменные без повторной сборки (2.3).

Каталог мастеров приведён в соответствии с рисунком 9.

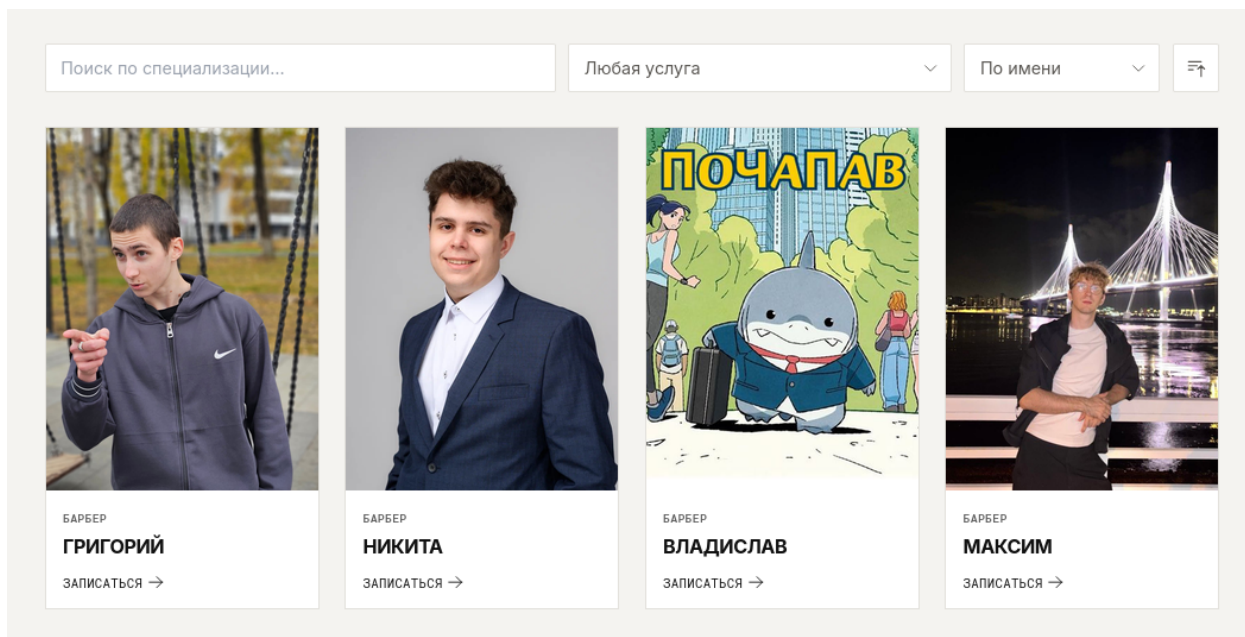


Рисунок 9 – Каталог мастеров (/masters)

Три элемента управления в верхней части формы соответствуют механизму выборки списков, описанному в подразделе 2.2: строка поиска по специализации, фильтр по оказываемой услуге и переключатель сортировки (по имени либо по цене, с изменением направления). Результат выдаётся постранично, не более двадцати карточек за запрос.

Страница записи к выбранному мастеру приведена в соответствии с рисунком 10.



БАРБЕР

Никита Кашинцев

Выберите услугу, дату и удобное время

01 УСЛУГА

Модельная стрижка	60 мин · 1000 Р
Наголо	30 мин · 600 Р
Удлиненная стрижка Стрижка только ножницами	60 мин · 1400 Р
Бокс/Полубокс	45 мин · 1000 Р
Моделирование бороды	30 мин · 600 Р
Обновление (стрижка только по бокам)	60 мин · 1200 Р
Детская стрижка	45 мин · 1200 Р
Стрижка под насадку	30 мин · 600 Р

02 ДАТА

СР 26 авг	ЧТ 27 авг	ПТ 28 авг	СБ 29 авг	ВС 30 авг	ПН 31 авг	ВТ 1 сен	СР 2 сен
ЧТ 3 сен	ПТ 4 сен	СБ 5 сен	ВС 6 сен	ПН 7 сен	ВТ 8 сен		

ОТЗЫВЫ

Пока нет отзывов

Рисунок 10 – Выбор услуги и даты на странице записи к мастеру

Форма приведена в отображении для узкого экрана и показывает результат адаптивной вёрстки (2.3): при ширине менее 640 пикселей карточки услуг и дат выстраиваются в одну колонку без горизонтальной прокрутки страницы. Цена каждой услуги – индивидуальная для мастера (master_service, 4.2), а перечень доступных дат формируется запросом свободных интервалов, показанным на рисунке 6.

Личный кабинет клиента приведён в соответствии с рисунком 11.

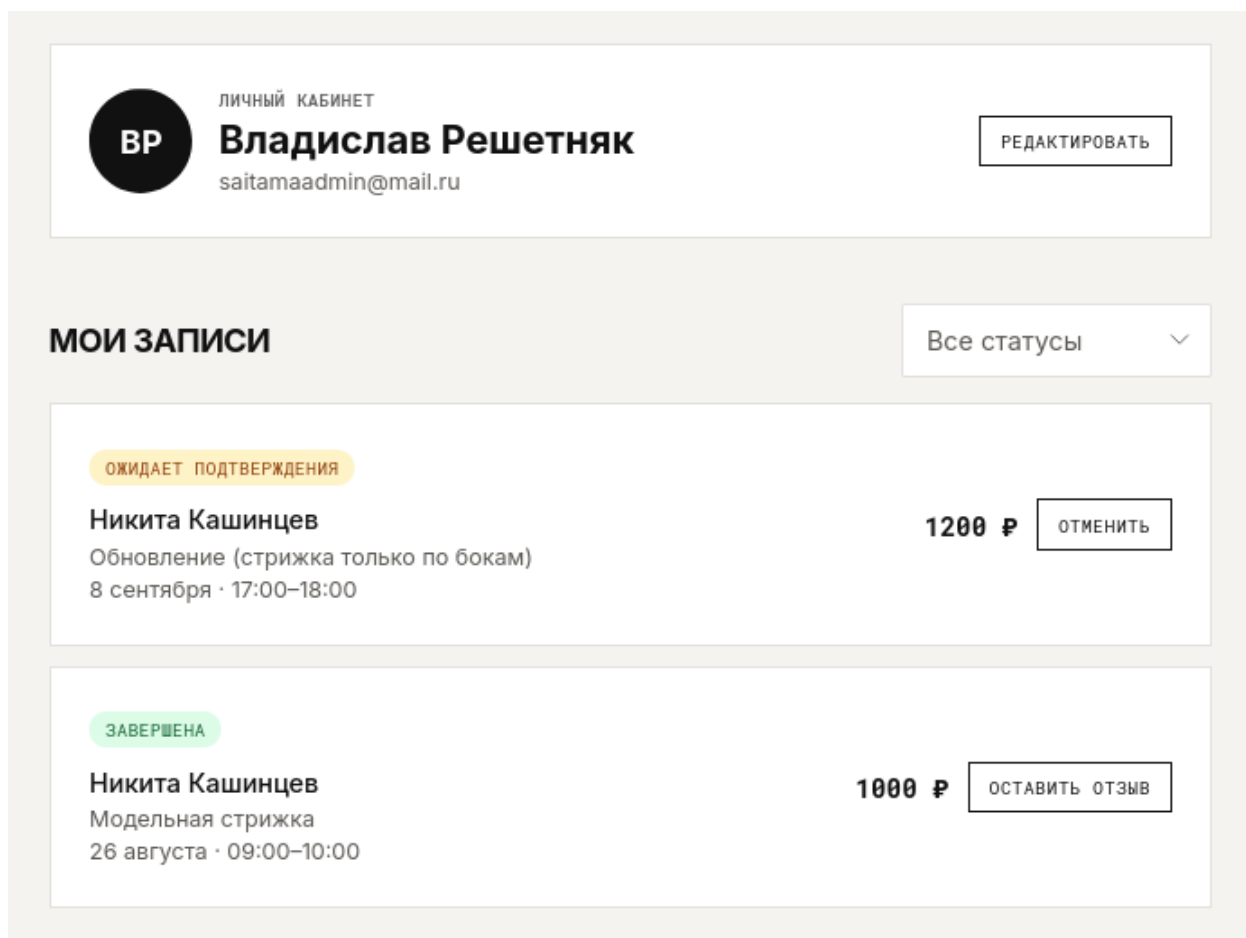


Рисунок 11 – Раздел «Мои записи» в личном кабинете клиента (/profile)

Состояние каждой записи отображается цветовой меткой и определяет доступный набор действий: запись в состоянии pending можно отменить, для записи в состоянии done предлагается оставить отзыв. Тем самым машина состояний, объявленная перечислимым типом appointment_status на уровне схемы данных (4.2), напрямую управляет составом элементов управления в интерфейсе.

Раздел «Статистика» панели администратора приведён в соответствии с рисунком 12.

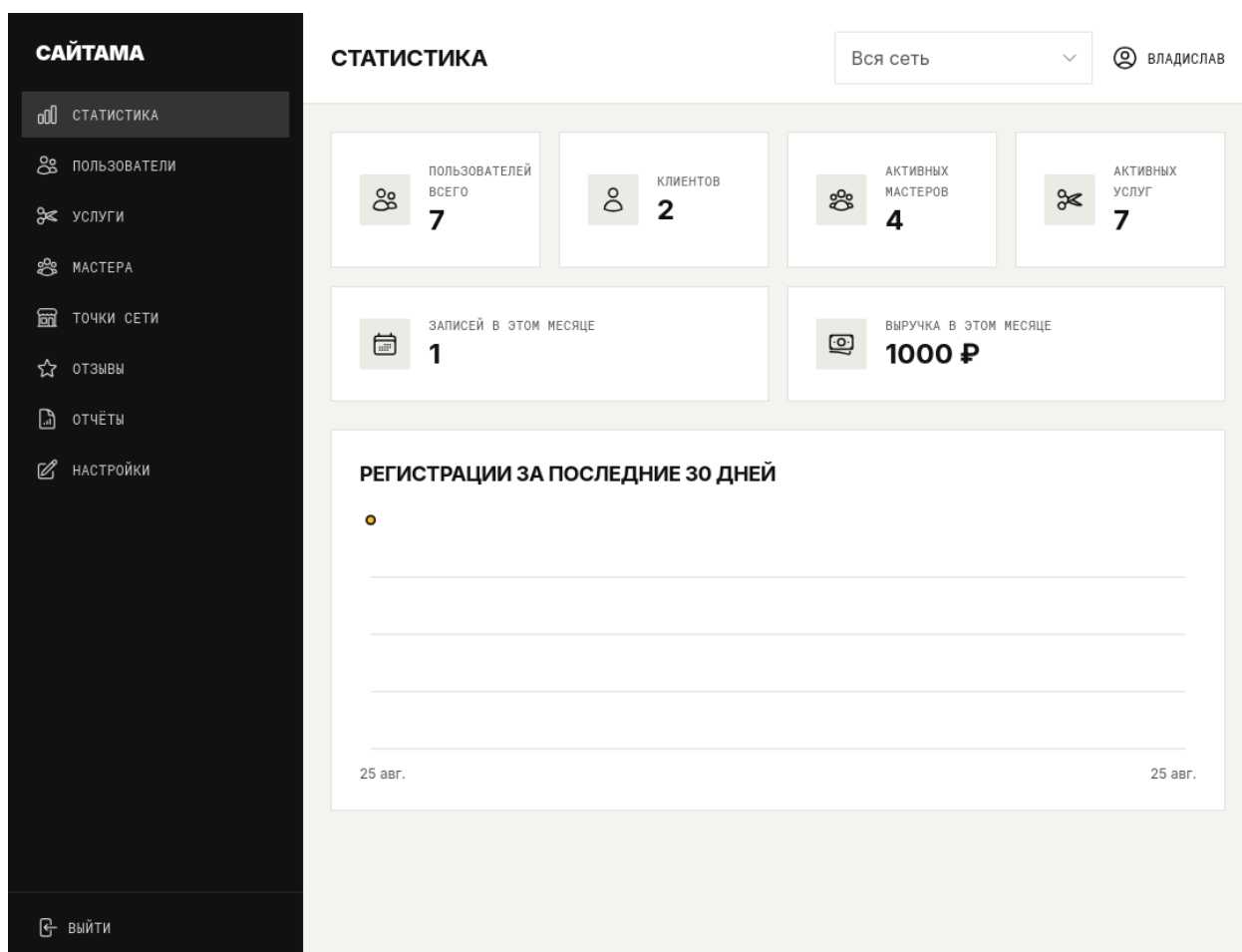


Рисунок 12 – Панель администратора (/admin), раздел «Статистика»

Слева расположена боковая панель навигации из восьми разделов, состав которой зависит от роли (2.3); в заголовке – переключатель точки сети, реализующий контроль контекста филиала (2.2). Показатели и график регистраций за 30 дней обновляются фоновым опросом с интервалом 30 секунд без участия пользователя.

Раздел «Отчёты» панели администратора приведён в соответствии с рисунком 13.

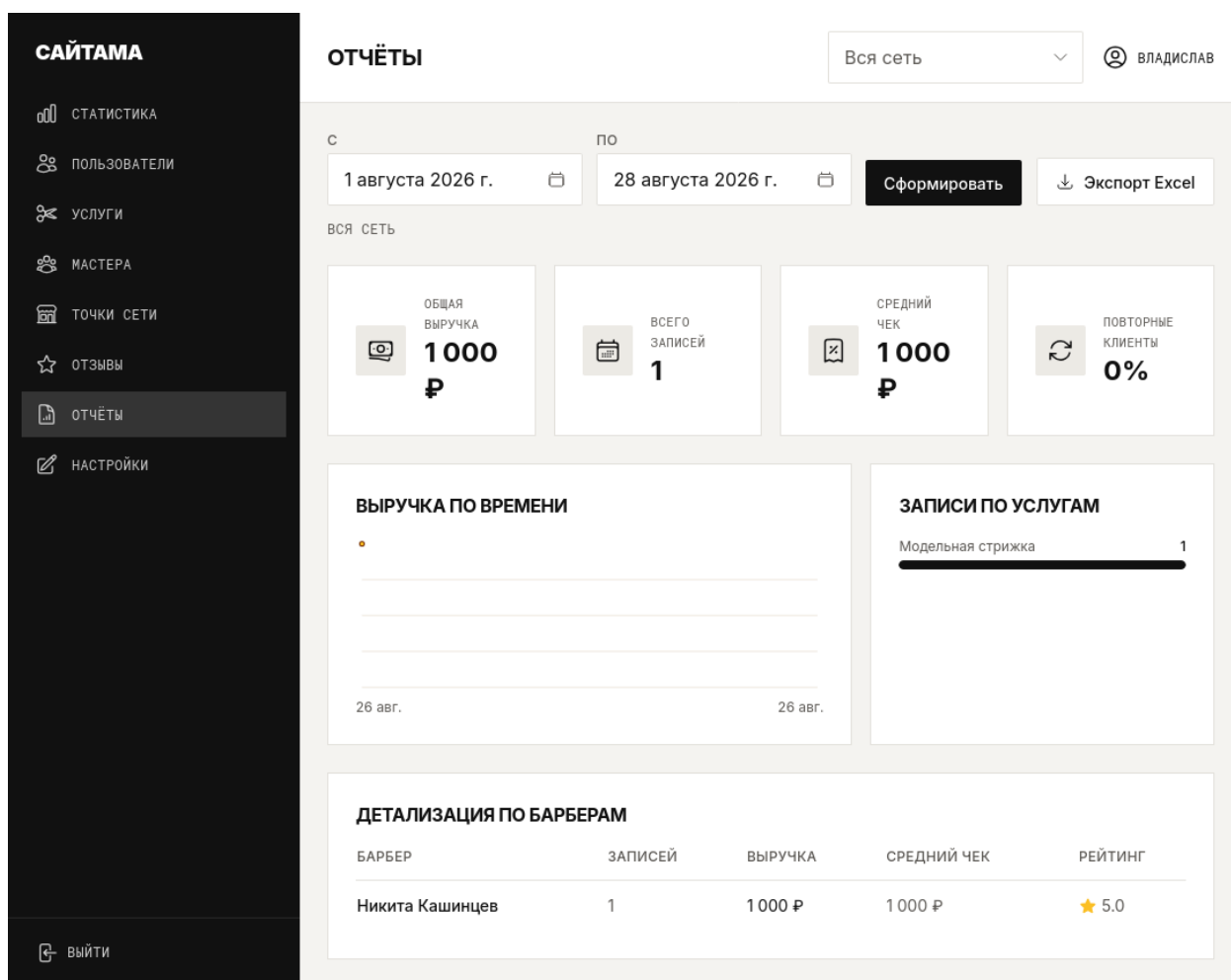


Рисунок 13 – Раздел «Отчёты» панели администратора

Отчёт строится за произвольный период и включает выручку, число записей, средний чек, долю повторных клиентов, распределение записей по услугам и детализацию по мастерам с выгрузкой в формат Excel. Форма реализует цель технического задания «формирование отчёта по выручке и загруженности без привлечения разработчика» (подраздел 2.2 технического задания) и целиком опирается на данные собственной базы данных проекта (5.5).

5.5 Подключение аналитики

В системе три независимых источника аналитики, которые не стоит путать между собой:

- **Веб-аналитика (Яндекс.Метрика)** – подключена и активна. Счётчик загружается отдельным статическим файлом (/metrika.js), а не инлайн-скриптом в index.html, – ради политики безопасности контента (CSP):

script-src остаётся 'self' без исключения 'unsafe-inline'. Использование раскрыто в §8 политики конфиденциальности (2.4).

- **Мониторинг ошибок (Sentry) – код готов, отключён по умолчанию.** Инициализация присутствует и на бэкенде, и на фронтенде, но выполняется только при заданном SENTRY_DSN/VITE_SENTRY_DSN.

- **Бизнес-аналитика – собственные отчёты по данным приложения.** Раздел «Статистика» (KPI-плитки, график регистраций за 30 дней) и раздел «Отчёты» (выручка и загруженность по дням/мастерам/услугам с экспортом в Excel) – оба на данных из собственной БД проекта, без внешнего сервиса. Реализуют цель технического задания «формирование отчёта по выручке и загруженности без привлечения разработчика» (подраздел 2.2); полное описание экранов – руководство пользователя, пункты 4.3.1 и 4.3.2.

5.6 Размещение в сети Интернет

Полный порядок развёртывания, включая перечень необходимых портов, подключение домена и TLS, первичную настройку и восстановление после сбоя, – предмет отдельного документа (руководство администратора, разделы 1–4) и здесь не повторяется. Кратко: система разворачивается как согласованный набор Docker-контейнеров [12] (база данных, backend, frontend, реверс-прокси Caddy [13] с автоматическим TLS-сертификатом Let's Encrypt, ежедневное резервное копирование), наружу из этой топологии открыты только порты 80/443 контейнера Caddy, обновление выполняется вручную одной командой (scripts/deploy.sh) без автоматического деплоя по CI (3.4).

ЗАКЛЮЧЕНИЕ

В ходе разработки создана и доведена до готовности к развёртыванию CMS «Сайтама» – веб-приложение для автоматизации онлайн-записи и операционной деятельности сети барбершопов.

Соответствие целям технического задания. Ключевые измеримые цели (подраздел 2.2 технического задания) обеспечены технически: двойное бронирование исключено ограничением целостности СУБД, а не организационно (раздел 5); индивидуальный прайс по мастеру, единое хранилище данных и отчёт по выручке/загруженности без участия разработчика – реализованы и доступны в интерфейсе администратора (5.4, 5.5). Часть целей (снижение доли неявок за счёт напоминаний, доля клиентов, выбирающих вход через VK ID) сформулирована как показатель, измеримый только на реальной эксплуатации.

Направления дальнейшего развития. Второй OAuth-провайдер и дополнительные каналы уведомлений можно добавить без изменения архитектуры (раздел 2 уже отделяет бизнес-логику сервисов от конкретного канала доставки/провайдера); горизонтальное масштабирование backend потребует пересмотра внутрипроцессных механизмов защиты от перебора паролей и ограничения частоты запросов (раздел 3) в пользу общего хранилища состояния; полноценный журнал аудита – отдельная таблица с записью каждой критичной операции, а не только попыток входа. Тестирование интерфейса с участием респондентов на подготовленных сценариях – ближайшая задача по клиентской части: часть предположений об очевидности шагов записи проверяема только наблюдением за реальными пользователями.

Итоговое состояние проекта, полный перечень функциональных требований и инструкции по эксплуатации – см. соответственно техническое задание, руководство пользователя и руководство администратора; весь комплект документации оформлен с учётом общих требований к программным документам [14].

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

- 1) FastAPI – официальная документация [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 29.08.2026).
- 2) SQLAlchemy 2.0 – официальная документация [Электронный ресурс]. – URL: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 29.08.2026).
- 3) PostgreSQL 16 – официальная документация [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 29.08.2026).
- 4) ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – М.: Стандартинформ, 2020.
- 5) ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. – М.: Изд-во стандартов, 1978.
- 6) Vue.js – официальная документация [Электронный ресурс]. – URL: <https://vuejs.org/> (дата обращения: 29.08.2026).
- 7) Pydantic – официальная документация [Электронный ресурс]. – URL: <https://docs.pydantic.dev/> (дата обращения: 29.08.2026).
- 8) Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных».
- 9) Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации».
- 10) ГОСТ Р ИСО/МЭК 15408-3-2008. Информационная технология. Методы и средства обеспечения безопасности. Критерии оценки безопасности информационных технологий. Часть 3. – М.: Стандартинформ, 2008.
- 11) Alembic – официальная документация [Электронный ресурс]. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 29.08.2026).
- 12) Docker Compose – официальная документация [Электронный ресурс]. – URL: <https://docs.docker.com/compose/> (дата обращения: 29.08.2026).
- 13) Caddy – официальная документация [Электронный ресурс]. –

URL: <https://caddyserver.com/docs/> (дата обращения: 29.08.2026).

14) ГОСТ 19.105-78. Единая система программной документации. Общие требования к программным документам. – М.: Изд-во стандартов, 1978.

ПРИЛОЖЕНИЕ А

(справочное)

Диаграммы последовательностей

Рисунки А.1–А.5 иллюстрируют порядок взаимодействия участников для ключевых бизнес-процессов системы, детализированных в карточках функциональных требований технического задания (подраздел 4.2).

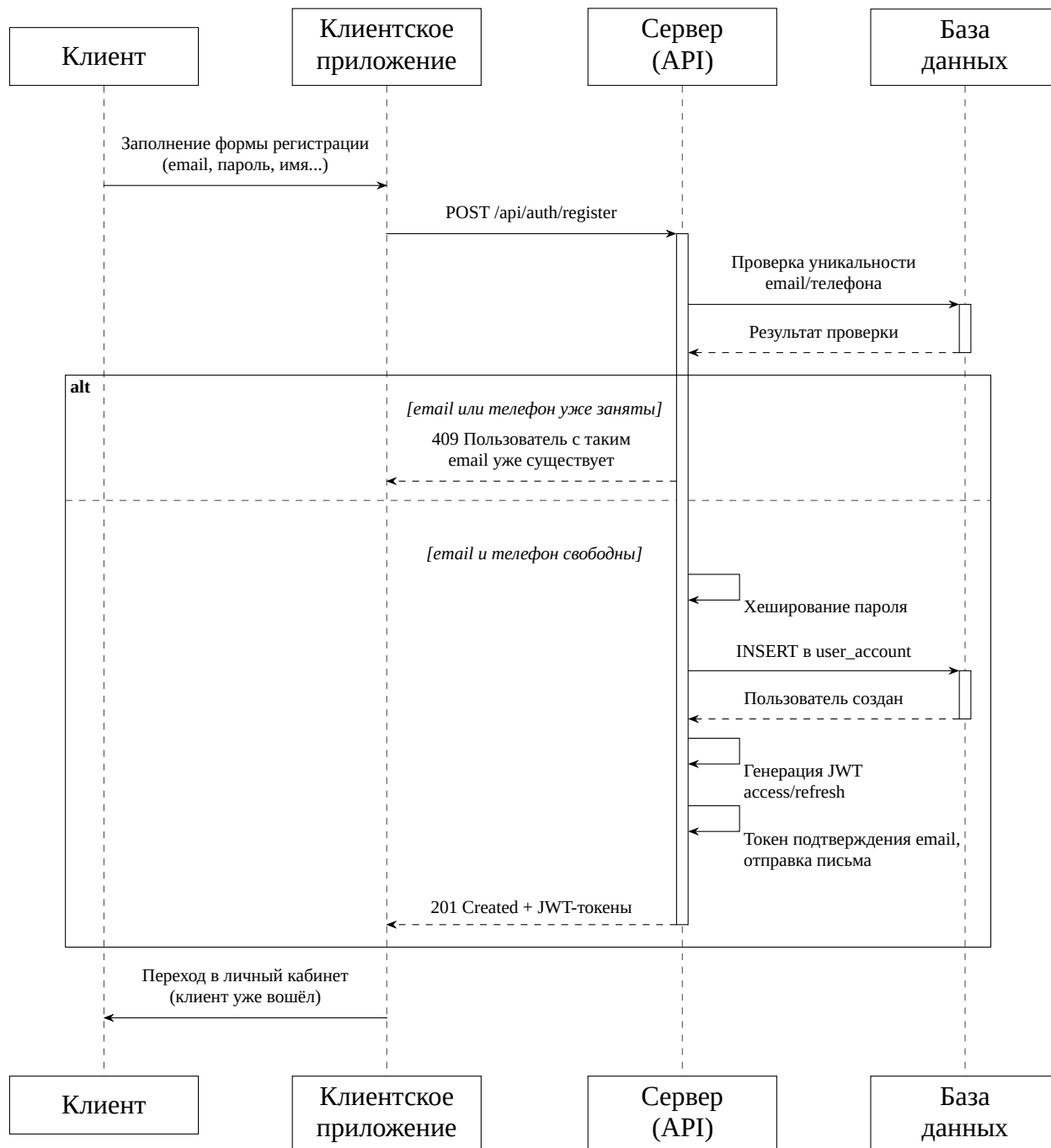


Рисунок А.1 – Диаграмма последовательностей бизнес-процесса «Регистрация» (ФТ-001)

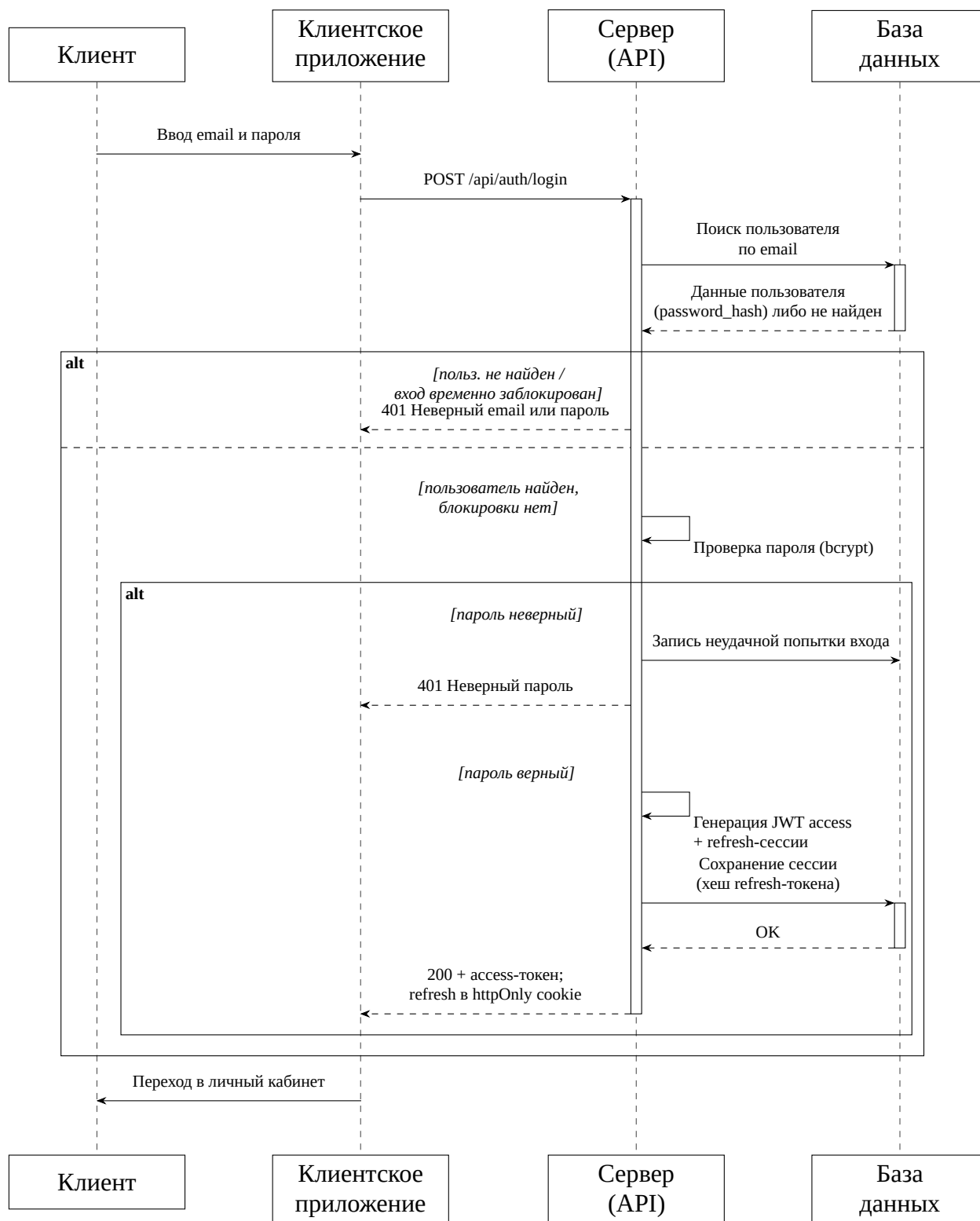


Рисунок А.2 – Диаграмма последовательностей бизнес-процесса «Вход по email и паролю»

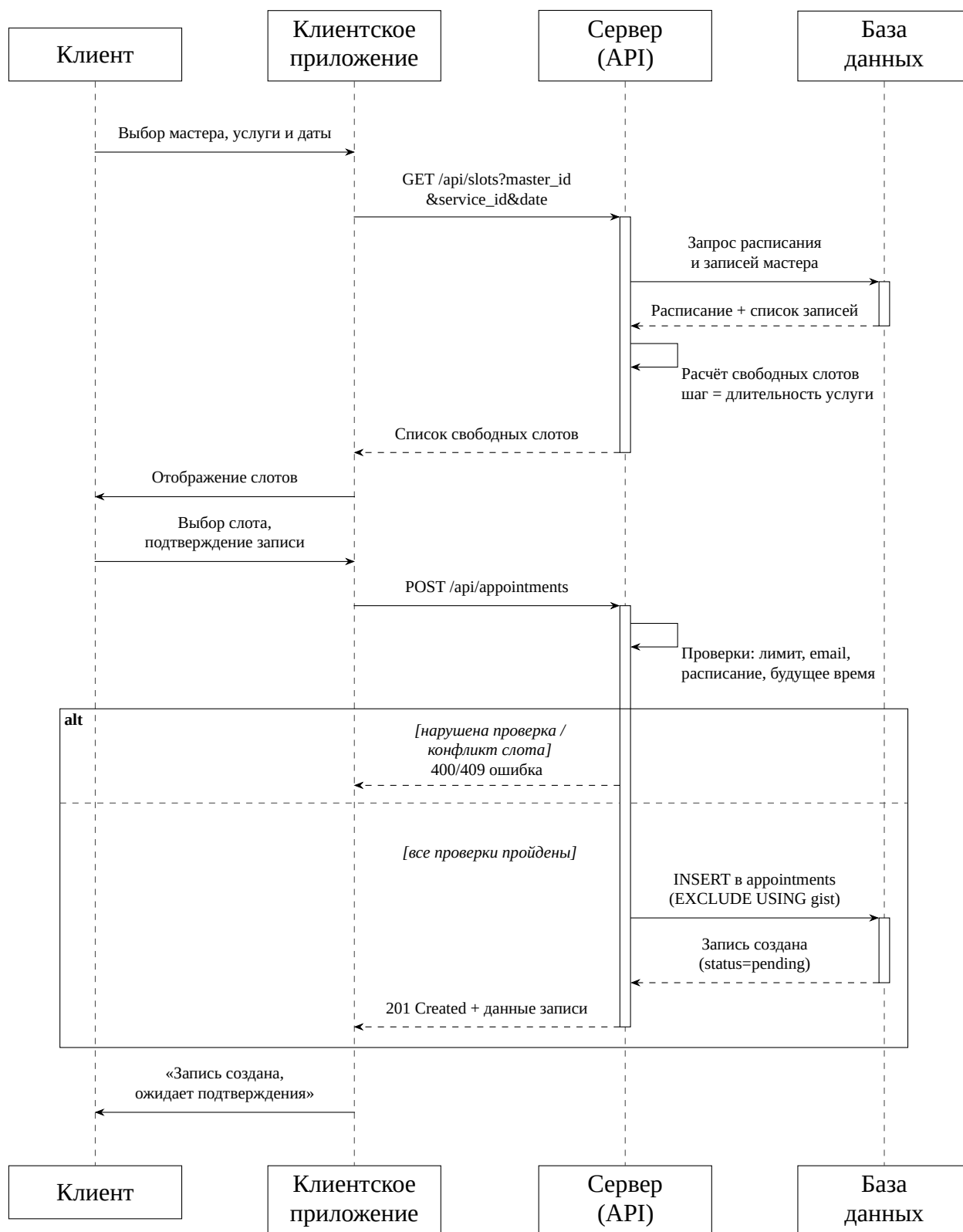


Рисунок А.3 – Диаграмма последовательностей бизнес-процесса «Онлайн-запись на приём» (ФТ-004, ФТ-005)

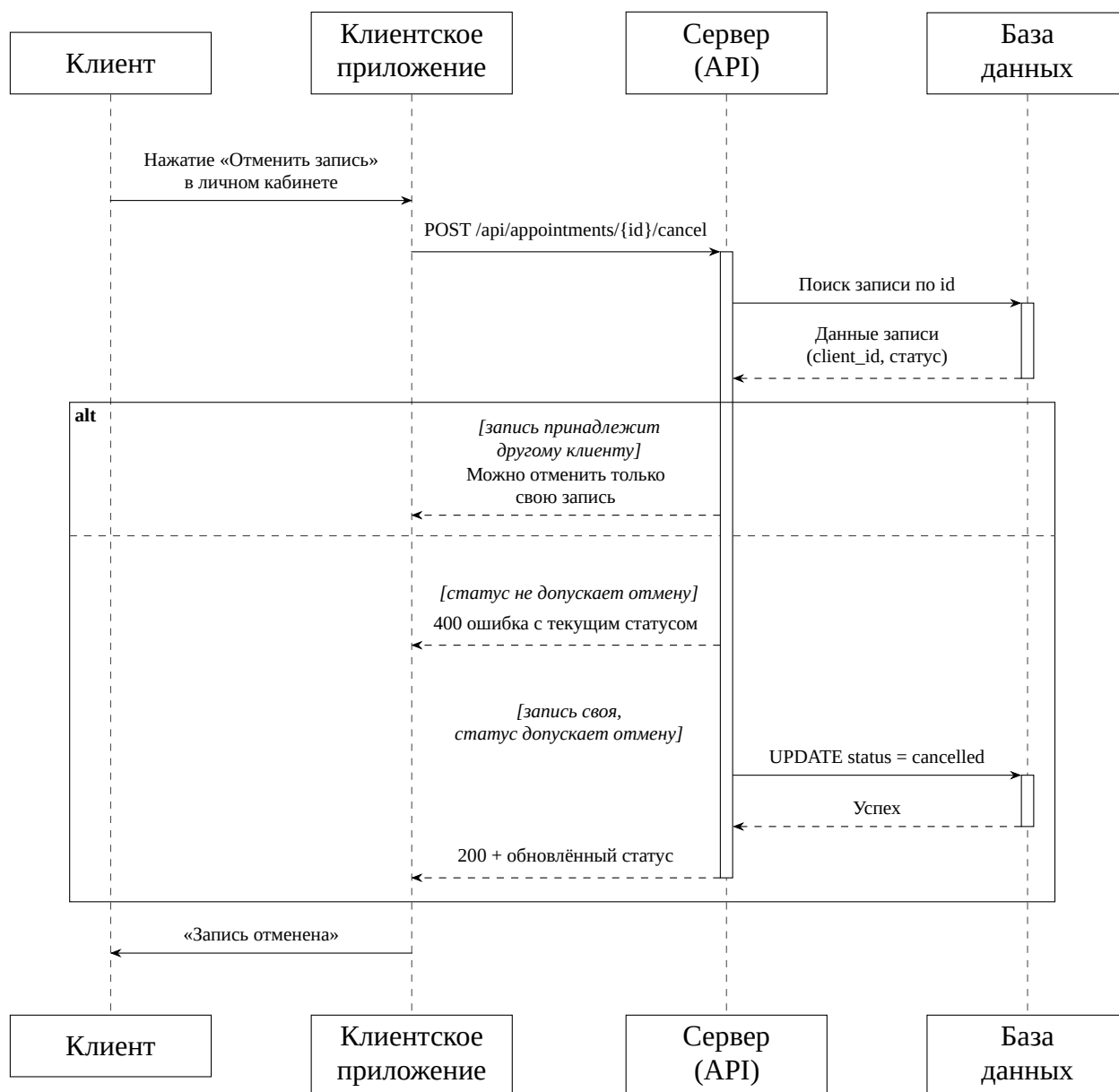


Рисунок А.4 – Диаграмма последовательностей бизнес-процесса «Отмена записи клиентом» (ФТ-006)

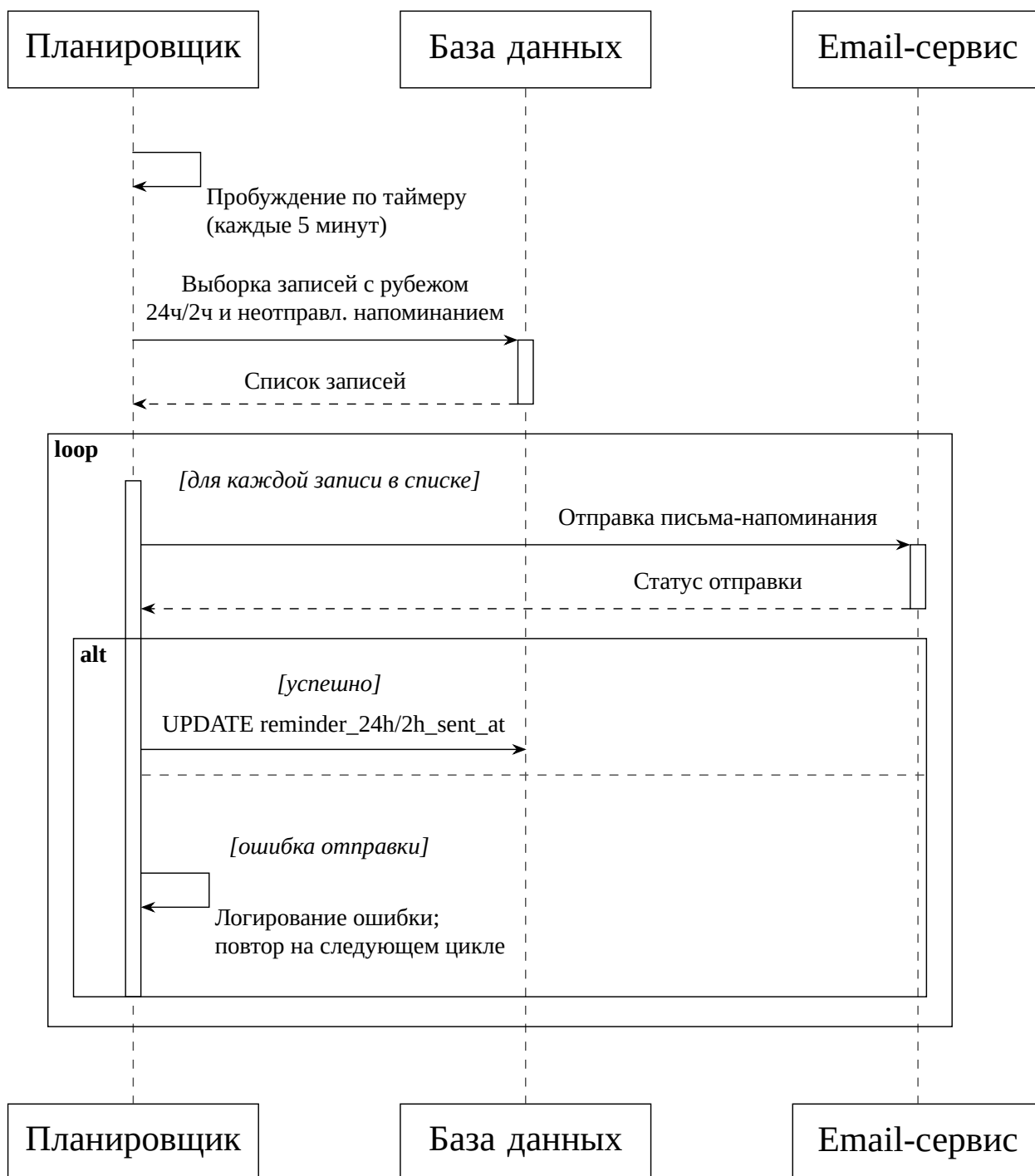


Рисунок А.5 – Диаграмма последовательностей бизнес-процесса
«Автоматическая отправка напоминаний» (ФТ-008)

ПРИЛОЖЕНИЕ Б

(обязательное)

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**СЫКТЫВКАРСКИЙ ЛЕСНОЙ ИНСТИТУТ (ФИЛИАЛ) ФЕДЕРАЛЬНОГО
ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО ОБРАЗОВАТЕЛЬНОГО
УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЛЕСОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМЕНИ С. М. КИРОВА» (СЛИ)**

ТРАНСПОРТНО-ТЕХНОЛОГИЧЕСКИЙ ФАКУЛЬТЕТ

КАФЕДРА ИНФОРМАЦИОННЫХ СИСТЕМ И ТЕХНОЛОГИЙ

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

к технологической (проектно-технологической) практике.
Web-технологии

Тема: разработка CMS для парикмахерских и барбершопов «Сайтама»

ПП.ТТФ.О.09.03.02.3.230176.ПЗ

на 56 листах

Выполнил: студент 3 курса
очной формы обучения
направления подготовки
бакалавриата «Информационные
системы и технологии»

_____ Решетняк В. А.
(подпись, дата)

Руководитель:
К. Э. Н., доцент

_____ Оганезова Н. А.
(подпись, дата)

Сыктывкар 2026

СОДЕРЖАНИЕ

1	Общие сведения	64
1.1	Полное наименование системы и приложения	64
1.2	Наименования заказчика и разработчика	64
1.3	Сроки разработки	64
1.4	Перечень документов, на которые ссылается ТЗ	64
2	Назначение и цели создания системы	66
2.1	Назначение системы	66
2.2	Цели создания	66
3	Характеристики объекта автоматизации	68
3.1	Описание предметной области	68
3.2	Границы автоматизации	69
3.3	Особенности реализации веб-приложения	70
4	Требования к системе	71
4.1	Требования к системе в целом	71
4.2	Требования к функциям (задачам), выполняемым системой	74
5	Требования к видам обеспечения	97
5.1	Информационное обеспечение	97
5.2	Лингвистическое обеспечение	108
5.3	Программное обеспечение	109
5.4	Техническое обеспечение	109
6	Состав и содержание работ по созданию системы	111
7	Порядок контроля и приёмки системы	112
	Виды, состав и методы испытаний	112
8	Требования к документированию	114
	Перечень документов	114

СПИСОК СОКРАЩЕНИЙ

API – Application Programming Interface – программный интерфейс взаимодействия компонентов системы.

CMS – Content Management System – система управления содержанием сайта.

CPU – Central Processing Unit – центральный процессор.

CRM – Customer Relationship Management – управление взаимоотношениями с клиентами.

CSRF – Cross-Site Request Forgery – межсайтовая подделка запроса.

CSS – Cascading Style Sheets – язык описания внешнего вида веб-страницы.

ER – Entity–Relationship – модель «сущность–связь», используемая для инфологического описания данных.

FK – Foreign Key – внешний ключ реляционной таблицы.

GB – гигабайт.

HTTP – HyperText Transfer Protocol – протокол передачи данных прикладного уровня; **HTTPS** – тот же протокол поверх шифрованного канала TLS.

ID – идентификатор объекта или учётной записи.

JSON – JavaScript Object Notation – текстовый формат обмена данными.

JWT – JSON Web Token – формат токена авторизации, используемый для access-токена сессии.

MVP – Minimum Viable Product – минимально жизнеспособный продукт; объём функций первой очереди системы.

OAuth – открытый протокол делегированной авторизации через внешнего поставщика учётных записей.

ORM – Object-Relational Mapping – отображение объектов языка программирования на строки реляционной таблицы.

PKCE – Proof Key for Code Exchange – расширение OAuth, защищающее обмен кода авторизации на токен от перехвата.

RAM – Random Access Memory – оперативная память.

RBAC – Role-Based Access Control – разграничение доступа на основе

ролей.

REST – Representational State Transfer – архитектурный стиль построения API поверх HTTP.

SEO – Search Engine Optimization – оптимизация страниц под поисковые системы.

SHA – Secure Hash Algorithm – семейство криптографических хеш-функций (в системе – SHA-256).

SMS – Short Message Service – служба коротких текстовых сообщений.

SMTP – Simple Mail Transfer Protocol – протокол отправки электронной почты.

SQL – Structured Query Language – язык запросов к реляционной СУБД.

SSD – Solid State Drive – твердотельный накопитель.

TLS – Transport Layer Security – протокол шифрования канала связи (HTTPS).

URL – Uniform Resource Locator – адрес ресурса в сети.

UTF-8 – Unicode Transformation Format, 8-bit – кодировка символов, используемая в системе.

UUID – Universally Unique Identifier – универсальный уникальный идентификатор.

VK ID – сервис единой учётной записи «ВКонтакте», используемый как внешний поставщик авторизации.

VPS – Virtual Private Server – виртуальный выделенный сервер.

XSS – Cross-Site Scripting – внедрение стороннего сценария в страницу приложения.

БД – база данных.

ПО – программное обеспечение.

СУБД – система управления базами данных (в проекте – PostgreSQL 16).

ТЗ – техническое задание.

ФЗ – федеральный закон.

ФТ – функциональное требование; карточки ФТ-001–ФТ-016 приведены в подразделе 4.2 настоящего ТЗ.

СПИСОК ТЕРМИНОВ

Владелец сети – пользователь с наибольшим объёмом прав: управляет всеми точками сети, каталогом услуг, содержимым сайта и назначением администраторов точек.

Запись – забронированный клиентом интервал времени у конкретного мастера на конкретную услугу в конкретной точке сети.

Карточка функционального требования – таблица единого формата, описывающая одну функцию системы: наименование, роли, входные и выходные данные, основной сценарий, альтернативные сценарии и обработку ошибок.

Мастер – сотрудник точки, оказывающий услуги клиентам; в системе – роль пользователя.

Мягкое удаление – перевод строки в неактивное состояние (поле deleted_at или is_active) вместо физического удаления, сохраняющий историю операций.

Слот – свободный интервал в расписании мастера, доступный для записи клиента.

Точка сети – отдельная парикмахерская или барбершоп в составе сети одного владельца – собственный адрес, часы работы, мастера и записи.

1 Общие сведения

1.1 Полное наименование системы и приложения

Наименование: CMS для парикмахерских и барбершопов.

Приложение: «Сайтама» (веб-приложение с клиентской и административной частью).

Понятия «парикмахерская» и «барбершоп» используются в настоящем ТЗ как взаимозаменяемые синонимы – предприятие сферы услуг стрижки и ухода за волосами/бородой (см. также раздел 3.1). Термин «грумминг-салон» в наименование системы не включён: в русскоязычной практике он обозначает услуги стрижки и ухода за животными, что не относится к сфере деятельности системы.

Система рассчитана на сеть из одной или нескольких точек обслуживания одного владельца: у сети общий каталог услуг и публичный сайт, но у каждой точки – собственный адрес, часы работы, мастера и записи (см. 3.1, 5.1.1 – сущность «Точка сети», и 5.1.2).

1.2 Наименования заказчика и разработчика

Заказчик: Решетняк Владислав Александрович, 335б, 3 курс.

Разработчик: Решетняк Владислав Александрович, 335б, 3 курс.

Руководитель проекта: Оганезова Нина Александровна.

1.3 Сроки разработки

Начало работ: 22.06.2026.

Окончание работ: 18.07.2026.

1.4 Перечень документов, на которые ссылается ТЗ

– ГОСТ 34.602-2020 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы»;

- ГОСТ 34.601-90 «Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания»;
- ГОСТ 19.201-78 «Единая система программной документации. Техническое задание. Требования к содержанию и оформлению»;
- Федеральный закон № 152-ФЗ «О персональных данных»;
- Федеральный закон № 149-ФЗ «Об информации, информационных технологиях и о защите информации».

2 Назначение и цели создания системы

2.1 Назначение системы

Система предназначена для автоматизации операционной деятельности барбершопов и парикмахерских, в том числе организованных в сеть из нескольких точек обслуживания одного владельца: ведения клиентской базы, управления расписанием мастеров, онлайн-записи клиентов, управления общим для сети каталогом услуг, формирования отчётов и ценообразованием, а также автоматической отправки писем-напоминаний о записи. Система обеспечивает кроссплатформенный доступ через веб-браузер, разграничивает права администратора точки (видит и обслуживает только свою точку) и владельца сети (видит и управляет всеми точками) и предоставляет оперативную аналитику по выручке, загруженности мастеров и популярности услуг – как по отдельной точке, так и по сети в целом.

Система автоматизирует следующие бизнес-процессы, выделенные в ходе исследования:

- 1) онлайн-запись клиента (Зап-1);
- 2) управление услугами и прайсом (Усл-2);
- 3) напоминания о записи (Увед-3);
- 4) формирование отчётов (Отчёт-4);
- 5) управление контентом сайта – настройки, отображаемые на главной странице и странице мастеров (Конт-5);
- 6) управление сетью точек обслуживания – открытие, закрытие, изменение адреса, телефона и часов работы каждой точки (Точ-6).

2.2 Цели создания

Каждая цель сформулирована с измеримым, проверяемым на приёмке показателем.

– **Сокращение времени обработки заявок.** Перевод записи в самообслуживание клиента: для уже зарегистрированного клиента время от открытия страницы записи до подтверждённой записи – не более 2 минут (против оценочных 5–10 минут при ручной обработке администратором

по телефону); регистрация нового клиента (однократная операция) в этот показатель не входит.

- **Устранение накладок и двойных записей.** 0 инцидентов двойного бронирования одного временного слота одного мастера – гарантируется ограничением целостности PostgreSQL (EXCLUDE USING gist), а не организационными мерами; проверяется нагрузочным тестированием конкурентных запросов (раздел 7).

- **Снижение доли пропущенных визитов.** Автоматические напоминания за 24 и 2 часа до визита (см. ФТ-008) призваны снизить долю визитов, забытых клиентом. Система не ведёт отдельного статуса «неявка» – факт оказания услуги мастер фиксирует вручную статусом done (см. 5.1.1), поэтому проверяемым на приёме показателем служит технический: не менее 95% подтверждённых записей получают оба напоминания (заполнены reminder_24h_sent_at и reminder_2h_sent_at) до наступления времени визита.

- **Гибкое ценообразование.** Возможность настроить индивидуальную стоимость услуг у конкретного мастера: коэффициентом мастера, умножающим базовую цену каждой оказываемой им услуги, либо точечным переопределением цены для одной услуги у одного мастера (см. ФТ-007). Формулировка уточнена: система не вычисляет цену автоматически по квалификации мастера – специализация мастера отображается администратору как справочная информация, но не участвует в расчёте.

- **Единое хранилище данных.** Полный отказ от бумажных журналов и разрозненных Excel-таблиц к моменту сдачи системы (100% операций записи, каталога услуг и клиентской базы ведётся в системе).

- **Аналитика для владельца.** Формирование отчёта по выручке, загруженности и топ-услугам за произвольный период – без привлечения разработчика, время формирования отчёта не более 3 секунд при объёме данных до 10 000 записей.

- **Упрощение авторизации клиентов.** Не менее 30% новых клиентов используют вход через VK ID вместо email-регистрации в течение первого месяца эксплуатации.

3 Характеристики объекта автоматизации

3.1 Описание предметной области

Объектом автоматизации является барбершоп, оказывающий услуги мужской стрижки, оформления бороды и сопутствующего ухода, в том числе организованный в сеть из нескольких точек обслуживания одного владельца. Пилотным объектом внедрения выступает барбершоп «Сайтама»; система проектируется технологически нейтрально к конкретному предприятию сферы услуг красоты и может быть использована другой парикмахерской или барбершопом без изменения кода – заменой каталога услуг и контента публичных страниц. Каждая точка сети имеет собственный адрес, часы работы и мастеров; каталог услуг и оформление публичного сайта – общие на всю сеть (см. 5.1.1, сущность «Точка сети»).

Предметная область охватывает:

- фиксацию и обработку заявок клиентов на запись к мастеру на конкретную услугу и время;
- ведение недельного расписания работы каждого мастера по дням недели, в пределах часов работы точки, к которой он относится (см. ФТ-011);
- управление каталогом услуг с индивидуальными ценами для каждого мастера;
- ведение клиентской базы (контактные данные, история визитов, способ авторизации); учётная запись клиента общая на всю сеть – повторная регистрация в каждой точке не требуется;
- автоматическую отправку писем-напоминаний о записи (за 24 и 2 часа до визита) и уведомления об изменении времени записи мастером/администратором;
- расчёт свободных временных слотов мастера на выбранную дату с учётом уже существующих записей (без технологического буфера между соседними записями, см. ФТ-004);
- оформление записи клиентом (доступно только вошедшим в систему пользователям, см. ФТ-005);
- приём отзывов клиентов о мастерах после завершённой услуги

(см. ФТ-014);

- управление контентом публичных страниц сайта (главная страница, страница мастеров, личный кабинет клиента);
- управление сетью точек обслуживания – открытие, закрытие, изменение адреса, телефона и часов работы каждой точки;
- формирование отчётов, включая аналитику загруженности мастеров, по отдельной точке и по сети в целом (см. ФТ-015);
- разграничение прав доступа между ролями: клиент, мастер, администратор точки, владелец сети.

3.2 Границы автоматизации

Вне границ автоматизации настоящей системы находятся следующие процессы – барбершоп продолжает вести их вне системы или с помощью сторонних инструментов:

- бухгалтерский и налоговый учёт (интеграция с 1С или иными бухгалтерскими системами не выполняется);
- складской учёт расходных материалов и косметических средств;
- расчёт и начисление заработной платы мастерам – система не ведёт кадровый учёт; настраиваемый у мастера коэффициент цены (coefficient, см. 5.1.1) используется только для расчёта стоимости услуг клиенту, а не вознаграждения мастера;
- приём онлайн-оплаты и возврат денежных средств – оплата услуг производится очно в барбершопе, система не выступает платёжным агрегатором (см. также ФТ-006);
- учёт фактической явки клиента отдельным статусом записи – терминальные статусы записи ограничены значениями «завершена» (услуга оказана, отмечается мастером) и «отменена» (см. 5.1.1); отдельного статуса «неявка» система не ведёт;
- CRM-функции за пределами базового профиля клиента: сегментация базы, email/SMS-маркетинговые рассылки, программы лояльности – не входят в MVP.

3.3 Особенности реализации веб-приложения

- централизованное хранение данных на сервере в PostgreSQL;
- многопользовательский доступ с разграничением прав по ролям (RBAC), включая ограничение видимости данных точкой сети для администратора;
- FastAPI отдаёт REST API, потребляемое Vue.js-фронтом;
- интеграция с внешним провайдером идентификации (VK ID);
- отправка писем напрямую через SMTP, без стороннего провайдера рассылок и без SMS-канала.

4 Требования к системе

4.1 Требования к системе в целом

4.1.1 Требования к структуре и функционированию системы

Клиент-серверная архитектура; компонентный состав системы приведён в таблице Б.1.

Таблица Б.1 – Компонентный состав системы

Компонент	Технология
Frontend	Vue.js + Tailwind CSS
Backend	FastAPI (Python)
СУБД	PostgreSQL 16+
ORM	SQLAlchemy 2.0 (синхронный) + Alembic (миграции)
Валидация данных	Pydantic
Напоминания о записи	фоновый asyncio-цикл в процессе backend (без выделенного воркера и брокера сообщений), см. ФТ-008

Система функционирует как веб-приложение, доступное через браузер по протоколу HTTPS, с обязательным интернет-соединением. Взаимодействие фронтенда и бэкенда осуществляется через REST API.

4.1.2 Требования к численности и квалификации персонала

Администратор системы – базовые навыки работы с административной панелью, специальной ИТ-подготовки не требуется.

Мастера и клиенты – не требуют специальной подготовки, интерфейс интуитивно понятен.

Обслуживание сервера (обновления, резервное копирование) – силами разработчика или на аутсорсе, специальной штатной единицы не

предусмотрено.

4.1.3 Требования к надёжности

Типовое для промышленных систем значение среднего времени наработки на отказ (8760 часов, 1 год) не подкрепляется методикой расчёта, так как не может быть проверено в отведённые на практику сроки. Установлены следующие требования:

- среднее время наработки на отказ – не менее 200 часов непрерывной работы в период опытной эксплуатации (соответствует периоду от развёртывания до защиты проекта, раздел 6, этап 5);
- время восстановления после сбоя – не более 30 минут;
- обязательное автоматическое резервное копирование базы данных не реже одного раза в сутки, хранение резервных копий не менее 7 дней;
- защита от потери данных при одновременной записи двух клиентов на один слот обеспечивается ограничением целостности PostgreSQL (EXCLUDE USING gist).

Критерии отказа. Отказом системы считается любое из следующих событий: недоступность системы (HTTP-ошибки 5xx на основных эндпоинтах либо отсутствие ответа) продолжительностью более 5 минут; потеря или необратимое искажение данных о записи или клиенте; невозможность выполнить любую из функций ФТ-001–ФТ-016 в течение более 10 минут подряд для более чем одного пользователя одновременно.

4.1.4 Требования к безопасности

- аутентификация пользователей: логин/пароль (email), а также OAuth 2.1 (PKCE) через VK ID;
- хранение паролей – только в виде хэша (bcrypt), хранение паролей в открытом виде запрещено;
- разграничение прав доступа по ролям: клиент, мастер, администратор точки, владелец сети; администратору дополнительно ограничена видимость данных его точкой (см. 3.1);
- авторизация запросов к API – JWT access-токен, 15 минут;

сессия входа (refresh) хранится на сервере как запись в таблице сессий с хешем токена в httpOnly-cookie – действительна 30 дней бездействия; смена/сброс пароля и блокировка пользователя аннулируют все его активные сессии;

- учёт попыток входа (успешных и неудачных, с IP-адресом и меткой времени) для защиты от подбора пароля – см. сущность «Попытка входа», 5.1.1; пять неудачных попыток по одному email за 15 минут временно блокируют вход по паролю для этого адреса;

- ограничение частоты запросов (rate limiting) по IP для функций, наиболее чувствительных к злоупотреблению: регистрация – не более 5 запросов в час, создание записи (онлайн-запись) – не более 20 запросов в минуту, оставление отзыва – не более 10 запросов в час; превышение лимита возвращает ошибку 429;

- персональные данные клиентов (ФИО, телефон, email) обрабатываются в соответствии с Ф3-152;

- защита от CSRF/XSS/SQL-инъекций стандартными средствами FastAPI и SQLAlchemy (параметризованные запросы, экранирование).

Определение способа входа. Отдельного поля-перечисления в модели пользователя нет: способ входа определяется по факту заполненности полей – наличие хеша пароля разрешает вход по email/паролю, наличие идентификатора VK ID – через VK ID (см. 5.1.1); поля независимы и могут быть заполнены одновременно. Первый вход через VK ID, чей email совпадает с email уже существующей учётной записи (созданной по паролю), автоматически привязывает VK ID к этой записи, а не создаёт вторую – подробнее см. ФТ-003.

4.1.5 Требования к эргономике и технической эстетике

Адаптивный дизайн, поскольку большая часть клиентов записывается с мобильных устройств.

Время отклика интерфейса – не более 300 мс на типовых операциях при штатной нагрузке (без учёта сетевых задержек); кратковременная деградация до 500 мс допускается только при пиковой нагрузке, соответствующей критерию нагрузочного теста (см. раздел 7), и не является нормой для

штатного режима работы.

Интуитивная навигация: запись на услугу – не более 4 шагов (выбор услуги, мастера, времени и подтверждение записи).

Единая цветовая и типографическая система на основе Tailwind CSS, применяемая единообразно на всех страницах.

Поддержка тёмной и светлой темы – опционально, не входит в MVP.

4.2 Требования к функциям (задачам), выполняемым системой

В таблицах Б.3–Б.18 приведены детализации ключевых функций системы (ФТ-001–ФТ-016). Таблица Б.2 демонстрирует полноту покрытия функциями всех четырёх ролей: «клиент», «мастер», «администратор точки» и «владелец сети», включая функции ролей «мастер» и «администратор» (ФТ-011–ФТ-016).

Таблица Б.2 – Функции системы по ролям

Функция	Клиент	Мастер	Админ.	Владелец
Регистрация, вход, восстановление пароля (ФТ-001, ФТ-002, ФТ-003, ФТ-010)	+	+	+	+
Просмотр слотов и запись (ФТ-004, ФТ-005)	+	–	–	–
Отмена/перенос своей записи (ФТ-006)	+	–	–	–
Просмотр своих записей, истории записей, выход из системы	+	+	+	+
Просмотр каталога услуг	–	–	+	+
Управление (создание/редактирование) каталога услуг (ФТ-007)	–	–	–	+
Получение писем-напоминаний о записи (ФТ-008)	+	+	–	–

Продолжение таблицы Б.2

Функция	Клиент	Мастер	Админ.	Владелец
Редактирование контента сайта (ФТ-009)	–	–	–	+
Управление собственным расписанием (ФТ-011)	–	+	+	+
Подтверждение/завершение/отмена записи клиента (см. 4.2.1)	–	+	+	+
Назначение ролей, создание профиля мастера (ФТ-012)	–	–	+	+
Отмена записи администратором (ФТ-013)	–	–	+	+
Оставление отзыва (ФТ-014)	+	–	–	–
Модерация отзывов (ФТ-014)	–	–	+	+
Аналитика загруженности мастеров, отчёты (ФТ-015)	–	–	+	+
Управление точками сети (ФТ-016)	–	–	–	+

Личный кабинет дополнительно обеспечивает функции, не требующие отдельной карточки ФТ ввиду тривиальности реализации: **выход из системы** (инвалидация текущей сессии – удаление refresh-токена на стороне сервера и очистка cookie на клиенте) и **просмотр истории записей** (список записей текущего пользователя с фильтром по статусу, включая завершённые/отменённые, а не только текущие).

Таблица Б.3 – Регистрация нового клиента (ФТ-001)

Параметр	Описание
Наименование функции	Регистрация нового клиента по email и паролю

Продолжение таблицы Б.3

Параметр	Описание
Идентификатор	ФТ-001
Цель и условие выполнения	Выполняется при обращении неаутентифицированного лица к форме регистрации на сайте барбершопа
Входные данные	Email, пароль, подтверждение пароля, имя, фамилия, необязательный телефон
Требования к обработке	1) Email должен соответствовать формату адреса электронной почты, приводится к нижнему регистру и должен быть уникальным среди активных учётных записей.
	2) Пароль – от 8 до 72 байт в UTF-8 (ограничение алгоритма хеширования bcrypt); требований к составу символов (регистр, цифры) нет.
	3) Телефон, если указан, – от 5 до 20 символов, только цифры и необязательные + () - и пробелы; специфического формата для российских номеров система не требует.
	4) Если email уже занят активной учётной записью – ошибка 409 «Пользователь с таким email уже существует»; аналогично для телефона.
	5) При успешной валидации создаётся клиент, пароль сохраняется в виде хеша (bcrypt), клиенту сразу выдаются JWT-токены (клиент считается вошедшим в систему). Параллельно генерируется токен подтверждения email (срок действия 24 часа) и отправляется письмо со ссылкой – подтверждение email не блокирует ни вход, ни какие-либо действия в системе (см. также ФТ-002).
	См. также ФТ-003 – альтернативный способ регистрации/входа через VK ID.

Продолжение таблицы Б.3

Параметр	Описание
Выходные данные	JWT access- и refresh-токены (клиент сразу вошёл в систему); запись в таблице user_account; письмо на email со ссылкой подтверждения

Таблица Б.4 – Подтверждение адреса электронной почты (ФТ-002)

Параметр	Описание
Наименование функции	Подтверждение email клиента; повторная отправка письма подтверждения
Идентификатор	ФТ-002
Цель и условие выполнения	Подтверждение – при переходе по ссылке из письма (действует и без входа в систему); повторная отправка – по кнопке в баннере, доступной только уже вошедшему пользователю с неподтверждённым email
Входные данные	Для подтверждения – уникальный токен из ссылки; для повторной отправки – входных данных не требуется, получатель определяется по текущей сессии
Требования к обработке	1) Проверка существования и срока действия токена (24 часа) при подтверждении; при недействительном или просроченном токене – ошибка «Токен подтверждения email недействителен или просрочен», без указания вошёл ли пользователь в систему.
	2) При успешном подтверждении – заполняется отметка времени подтверждения email, токен помечается использованным; новых токенов сессии подтверждение не выдаёт – пользователь уже был или не был вошедшим независимо от этого действия.

Продолжение таблицы Б.4

Параметр	Описание
	3) Повторная отправка письма для уже подтверждённого email – ошибка 400 «Email уже подтверждён» (это отдельная проверка при нажатии «Отправить письмо ещё раз», а не результат перехода по использованной ссылке).
	4) Повторная отправка ограничена – не более 3 запросов в час на пользователя, иначе ошибка 429 «Слишком много запросов. Попробуйте позже.»; каждая повторная отправка аннулирует ранее выданные, ещё не использованные токены этого пользователя.
Выходные данные	Признак успешного подтверждения либо соответствующее сообщение об ошибке

Таблица Б.5 – Авторизация через VK ID (OAuth 2.1, PKCE) (ФТ-003)

Параметр	Описание
Наименование функции	Авторизация клиента через сторонний OAuth-провайдер
Идентификатор	ФТ-003
Цель и условие выполнения	Выполняется при нажатии клиентом кнопки «Войти через VK»; кнопка показывается, только если на сервере включена интеграция с VK ID
Входные данные	Код авторизации (authorization code) и code_verifier, возвращённые провайдером после согласия пользователя (обмен по протоколу OAuth 2.1 с PKCE, метод S256)
Требования к обработке	1) Система выполняет обмен кода на access-токен провайдера с проверкой code_verifier.

Продолжение таблицы Б.5

Параметр	Описание
	2) Получает профиль пользователя (идентификатор VK, имя, email при наличии).
	3) Ищет пользователя по идентификатору VK; если не найден – ищет по email (если VK его вернул): если учётная запись с таким email уже существует (создана по паролю) – идентификатор VK привязывается к ней, а email, если ещё не был подтверждён, считается подтверждённым; если нет – создаётся новая учётная запись с ролью «клиент», без пароля.
	4) Проверка, что учётная запись не заблокирована – иначе отказ.
	5) Выдаёт собственные JWT-токены (как при обычном входе).
	6) Ошибки провайдера (отказ пользователя, недействительный/просроченный код, сбой обмена токена) приводят к редиректу на страницу входа с кодом ошибки в адресе; интерфейс показывает пользователю соответствующее сообщение.
Выходные данные	JWT access- и refresh-токены; учётная запись клиента (создана либо найдена/дополнена)
Особенности	Email для клиентов, авторизованных через VK, может отсутствовать – в этом случае поле email остаётся пустым, пока клиент не укажет его вручную (в профиле либо на шаге записи, см. ФТ-005), а функции, требующие email (в т.ч. запись к мастеру), до этого момента недоступны.

Таблица Б.6 – Получение списка свободных временных слотов (ФТ-004)

Параметр	Описание
Наименование функции	Расчёт доступных для записи временных слотов
Идентификатор	ФТ-004
Цель и условие выполнения	Выполняется при выборе клиентом мастера, услуги и даты на странице онлайн-записи
Входные данные	master_id, service_id, date
Требования к обработке	1) Получить рабочее расписание мастера на день недели, соответствующий указанной дате; если для этого дня недели нет строки расписания либо день отмечен нерабочим – вернуть пустой массив слотов.
	2) Получить длительность выбранной услуги.
	3) Пройти рабочее окно мастера (от начала до конца смены) шагом, равным длительности выбранной услуги; интервал, пересекающийся с уже существующей записью мастера, пропускается, и следующая попытка начинается сразу после окончания этой записи – шаг не выровнен по фиксированной сетке.
	4) Слоты, целиком относящиеся к прошедшему времени (при запросе на сегодня), не возвращаются.
Выходные данные	Массив доступных слотов в формате {start_time, end_time}; при отсутствии свободных слотов – пустой массив

Продолжение таблицы Б.6

Параметр	Описание
Особенности	Слоты вычисляются динамически при каждом запросе, не хранятся отдельной сущностью. Технологического буфера между соседними записями одного мастера система не применяет – следующая запись может начинаться сразу по окончании предыдущей. Клиентское приложение на шаге «Дата» предлагает выбор только из ближайших 14 дней – это ограничение интерфейса, backend его не проверяет и рассчитывает слоты на любую будущую дату, если запрос придёт в обход интерфейса.

Таблица Б.7 – Создание записи (бронирование) (ФТ-005)

Параметр	Описание
Наименование функции	Создание записи клиента к мастеру
Идентификатор	ФТ-005
Цель и условие выполнения	Выполняется вошедшим в систему клиентом после выбора мастера, услуги и слота; незарегистрированным посетителям и гостям функция недоступна – запись без учётной записи система не создаёт
Входные данные	master_id, service_id, start_time; клиент определяется из токена сессии (client_id отдельно не передаётся)
Требования к обработке	1) Проверка лимита: не более 5 одновременных активных (не завершённых и не отменённых) записей у клиента – иначе ошибка 400.
	2) Проверка, что у клиента заполнен email – иначе ошибка 400 «Укажите email в профиле, чтобы оформить запись» (актуально для клиентов, вошедших через VK ID без email, см. ФТ-003).

Продолжение таблицы Б.7

Параметр	Описание
	3) Проверка существования и активности мастера и услуги, что мастер оказывает выбранную услугу, и что клиент не пытается записаться сам к себе (для случая, когда мастер бронирует у другого мастера).
	4) Проверка, что start_time – в будущем; иначе ошибка 400 «Нельзя записаться на прошедшее время».
	5) Расчёт end_time = start_time + длительность услуги; расчёт итоговой цены – price_override у пары «мастер – услуга», если задан, иначе цена услуги × коэффициент мастера (округление до копеек).
	6) Проверка, что интервал [start_time, end_time) укладывается в рабочее расписание мастера на этот день недели и не выходит за часы работы точки – иначе ошибка 400 с указанием причины (мастер не работает в этот день / запись выходит за рамки его рабочего времени).
	7) Предварительная проверка отсутствия пересечения с другими записями мастера, затем попытка INSERT в таблицу appointment с ограничением EXCLUDE USING gist по пересечению временного диапазона для одного мастера – страхует от гонки двух одновременных запросов.
	8) При конфликте (нарушение EXCLUDE-ограничения) – возврат ошибки 409 «Выбранное время уже занято».

Продолжение таблицы Б.7

Параметр	Описание
	9) При успехе – запись создаётся со статусом pending («ожидает подтверждения»); подтверждает её мастер или администратор вручную (см. 4.2.1). Отдельного письма-подтверждения о создании записи система не отправляет.
Выходные данные	Объект созданной записи (id, статус pending, время, мастер, услуга, итоговая цена); либо сообщение об ошибке 400/404/409

Таблица Б.8 – Отмена записи клиентом (ФТ-006)

Параметр	Описание
Наименование функции	Отмена клиентом собственной записи
Идентификатор	ФТ-006
Цель и условие выполнения	Выполняется клиентом из личного кабинета для собственной записи в статусе «ожидает подтверждения» или «подтверждена»
Входные данные	appointment_id
Требования к обработке	1) Проверка, что запись принадлежит обратившемуся клиенту – иначе ошибка 403 «Можно отменить только свою запись».
	2) Проверка, что текущий статус записи допускает переход в «отменена» (pending или confirmed) – иначе ошибка 400 с указанием текущего статуса. Ограничения по времени до визита нет – клиент может отменить запись в любой момент вплоть до её начала, штрафов и дедлайна отмены система не применяет.

Продолжение таблицы Б.8

Параметр	Описание
	3) Статус записи меняется на cancelled, слот освобождается.
	4) Онлайн-оплата в системе не предусмотрена (см. 3.2) – механизм возврата денежных средств не требуется.
	5) Перенести время записи клиент самостоятельно не может – эта операция доступна только мастеру или администратору (см. ФТ-013).
Выходные данные	Обновлённый статус записи

Таблица Б.9 – Управление каталогом услуг и индивидуальным прайсом мастера (ФТ-007)

Параметр	Описание
Наименование функции	Создание/редактирование услуги каталога и настройка её цены у мастеров
Идентификатор	ФТ-007
Цель и условие выполнения	Создание, редактирование и скрытие услуги – только владельцем сети (каталог – общий актив всей сети); привязка услуги к мастеру и её индивидуальная цена у этого мастера – владельцем сети либо администратором точки, к которой относится мастер
Входные данные	Название услуги, описание (необязательно), базовая цена (> 0), длительность в минутах (> 0); для привязки к мастеру – master_id, необязательное точечное переопределение цены (price_override)
Требования к обработке	1) Валидация: цена > 0 , длительность – целое число минут > 0 (кратность 15 минутам не требуется), название – от 2 до 200 символов.

Продолжение таблицы Б.9

Параметр	Описание
	2) Сохранение услуги в таблице service.
	3) Привязка услуги к мастеру – таблица master_service; итоговая цена услуги у мастера = price_override, если он задан явно, иначе – базовая цена услуги, умноженная на общий коэффициент цены мастера (coefficient, см. 5.1.1) и округлённая до копеек.
	4) Удаление привязки услуги к мастеру – отдельным действием (без ограничений). Скрытие самой услуги из каталога (is_active = false) – мягкое, услугу можно вернуть в каталог повторным редактированием; физическое удаление услуги, на которую есть привязки к мастерам или записи, системой не предусмотрено.
Выходные данные	Карточка услуги; записи в master_service; обновлённый каталог, немедленно доступный через публичный API

Таблица Б.10 – Автоматическая отправка писем-напоминаний о записи (ФТ-008)

Параметр	Описание
Наименование функции	Формирование и отправка клиенту письма-напоминания о предстоящей записи
Идентификатор	ФТ-008
Цель и условие выполнения	Выполняется автоматически фоновым процессом внутри backend, опрашивающим базу данных каждые 5 минут (см. 4.1.1); отдельного воркера или очереди сообщений нет

Продолжение таблицы Б.10

Параметр	Описание
Входные данные	Записи в статусе «ожидает подтверждения» или «подтверждена», у которых до начала визита остался 24-часовой либо 2-часовой рубеж и соответствующее напоминание ещё не отправлено
Требования к обработке	1) На каждом проходе выбираются записи, которым пора получить напоминание «за 24 часа» (до визита от 2 до 24 часов, ещё не отправлено) и отдельно – «за 2 часа» (до визита не более 2 часов, ещё не отправлено); если проход был пропущен (например, backend был недоступен) и запись уже попала в 2-часовое окно, ей отправляется только напоминание «за 2 часа», без дублирования.
	2) Формирование текста письма (тема, услуга, мастер, дата и время) и отправка на email клиента через SMTP.
	3) Ошибка сборки письма для одной записи не останавливает рассылку остальным в этом же проходе; ошибка отправки через SMTP не приводит к повторной попытке – фиксируется в журнале сервера.
	4) После попытки отправки в поле reminder_24h_sent_at либо reminder_2h_sent_at записи проставляется метка времени – она же служит признаком «уже отправлено» для следующих проходов.
Выходные данные	Письмо клиенту на email; обновлённая метка времени отправки в записи

Продолжение таблицы Б.10

Параметр	Описание
Особенности	Уведомления мастеру, отдельные письма-подтверждения при создании записи и SMS-канал системой не реализованы (см. 3.3); мастер и администратор видят новые/изменённые записи непосредственно в личном кабинете/панели (4.2.1).

Таблица Б.11 – Редактирование контента публичных страниц (главная, мастера) (ФТ-009)

Параметр	Описание
Наименование функции	Редактирование содержания и оформления сайта (тексты, цвета, шрифты, изображения, SEO)
Идентификатор	ФТ-009
Цель и условие выполнения	Выполняется только владельцем сети в разделе «Настройки» – редактирование выполняется прямо на самой главной странице сайта, отдельной формы нет
Входные данные	Тексты блоков главной страницы, выбор цветовой темы и пары шрифтов, файлы изображений (логотип, hero-фото), заголовков вкладки браузера, описание для поисковых систем, favicon
Требования к обработке	1) Валидация текстовых полей на длину.
	2) Загрузка изображений: допустимые форматы JPEG/PNG/WebP/GIF, ограничение по размеру файла – не более 5 МБ; проверка типа выполняется по заголовку Content-Type файла, содержимое (в т.ч. разрешение) не анализируется.

Продолжение таблицы Б.11

Параметр	Описание
	3) Замена изображения сохраняет новый файл под новым именем и обновляет ссылку на него в настройках; предыдущий файл с диска не удаляется.
	4) Обновление единственной строки конфигурации (таблица site_setting, содержимое хранится в поле content в формате JSON).
Выходные данные	Обновлённые настройки, немедленно отражающиеся на публичных страницах
Особенности	Адрес и часы работы на публичном сайте (подвал, блок «Наши салоны») эта функция не редактирует – они берутся из данных точек сети (см. ФТ-016).

Таблица Б.12 – Восстановление пароля (ФТ-010)

Параметр	Описание
Наименование функции	Восстановление доступа к учётной записи по забытому паролю
Идентификатор	ФТ-010
Цель и условие выполнения	Выполняется при обращении к форме «Забыли пароль?» неаутентифицированным пользователем; для учётной записи, созданной только через VK ID (без пароля), функция бессмысленна – восстанавливать нечего
Требования к обработке	1) По указанному email ищется учётная запись; независимо от результата поиска клиенту показывается одно и то же сообщение (не раскрывает, зарегистрирован ли адрес).

Продолжение таблицы Б.12

Параметр	Описание
	2) При наличии учётной записи – генерируется одноразовый токен сброса (срок действия 30 минут), отправляется письмо со ссылкой; лимит – не более 3 запросов в час на адрес.
	3) По переходу на форму нового пароля с валидным токеном – установка нового password_hash (валидация как в ФТ-001), все ранее выданные JWT-токены пользователя аннулируются (принудительный выход со всех устройств), токен сброса аннулируется.
Выходные данные	Письмо со ссылкой сброса; после установки нового пароля – сообщение об успехе и предложение войти

Таблица Б.13 – Управление расписанием мастера (ФТ-011)

Параметр	Описание
Наименование функции	Ведение недельного рабочего графика мастера
Идентификатор	ФТ-011
Цель и условие выполнения	Мастер редактирует собственное расписание; администратор точки/владелец сети – расписание мастеров своей точки (владелец – любой точки)
Входные данные	master_id, день недели (0 – понедельник, ..., 6 – воскресенье), start_time, end_time, признак is_working

Продолжение таблицы Б.13

Параметр	Описание
Требования к обработке	1) Расписание – одна строка на каждый день недели (не более семи на мастера), без привязки к конкретной календарной дате; отпуска, больничные и разовые переносы смены отдельными типами дня система не выражает – на такие дни день недели временно переводится в is_working = false вручную.
	2) При is_working = true – валидация start_time < end_time, а также попадания смены в часы работы точки, к которой относится мастер – иначе ошибка «Расписание должно быть в пределах времени работы салона (ЧЧ:ММ–ЧЧ:ММ)».
	3) При is_working = false – поля start_time/end_time не проверяются, день недели полностью недоступен для записи (ФТ-004 вернёт пустой список слотов).
	4) Изменение расписания не проверяется на конфликт с уже существующими записями мастера на этот день недели – система не отменяет их и не предупреждает администратора; если после сужения смены запись выходит за её пределы, ответственность за перенос/отмену такой записи – на мастере/администраторе (ФТ-013).
Выходные данные	Обновлённая строка расписания на выбранный день недели

Таблица Б.14 – Назначение ролей и создание профиля мастера (ФТ-012)

Параметр	Описание
Наименование функции	Изменение роли пользователя и связанное управление профилем мастера
Идентификатор	ФТ-012

Продолжение таблицы Б.14

Параметр	Описание
Цель и условие выполнения	Выполняется администратором точки (роли client/master и мастера только своей точки) либо владельцем сети (без ограничений) в разделе «Пользователи»
Входные данные	user_id, новая роль (client/master/admin); при назначении admin – точка, которой будет управлять пользователь; при создании профиля мастера – точка (для владельца сети обязательна, для администратора точки подставляется автоматически)
Требования к обработке	1) Проверка, что запрашивающий имеет роль admin/owner; назначение роли admin доступно только owner (роль owner через смену роли не назначается и не снимается).
	2) Назначение роли admin требует, чтобы пользователю уже была назначена точка (см. отдельное действие «привязка точки») – иначе ошибка 400 «Сначала назначьте пользователю точку».
	3) Смена роли на master сама по себе профиль мастера не создаёт – это отдельное действие «Создать профиль мастера», доступное после смены роли; если профиль ранее существовал и был деактивирован – он реактивируется вместо создания нового, чтобы не терять историю записей и отзывов; попытка создать профиль повторно активному мастеру – ошибка 409 «Профиль мастера уже существует».

Продолжение таблицы Б.14

Параметр	Описание
	4) При смене роли на любую, кроме master, – существующий профиль мастера деактивируется (is_active = false), не удаляется.
Выходные данные	Обновлённый пользователь; профиль мастера (создан/реактивирован/деактивирован)

Таблица Б.15 – Управление статусом и переносом записи мастером/администратором (ФТ-013)

Параметр	Описание
Наименование функции	Подтверждение, завершение, отмена и перенос времени записи со стороны мастера или администратора
Идентификатор	ФТ-013
Цель и условие выполнения	Выполняется мастером для своих записей либо администратором/владельцем сети без этого ограничения; в отличие от ФТ-006 (отмена клиентом), доступны также подтверждение, завершение и перенос времени
Входные данные	appointment_id; для смены статуса – новый статус (confirmed/done/cancelled); для переноса – новый start_time
Требования к обработке	1) Проверка, что мастер меняет статус/переносит только свою запись – иначе 403; администратор/владелец – без этого ограничения.
	2) Переход статуса проверяется по фиксированной схеме: pending → confirmed → done; cancelled достижим из pending или confirmed; done и cancelled – терминальные, дальнейших переходов из них нет. Переход, не входящий в допустимую схему, – ошибка 400.

Продолжение таблицы Б.15

Параметр	Описание
	3) Перенос времени допустим только для записи в статусе pending или confirmed, только на будущее время, только у того же мастера и той же услуги – смена мастера или услуги требует отмены и повторного бронирования (ФТ-005); новое время проверяется теми же правилами, что и при создании (в пределах рабочего расписания мастера, отсутствие пересечения с другими записями).
	4) Ни привязки к моменту начала визита, ни ограничения по времени до визита ни у одной из операций нет – в т.ч. отметить запись завершённой можно до фактического начала визита.
	5) Онлайн-оплата в системе не предусмотрена (см. 3.2) – механизм возврата денежных средств не требуется.
	6) При переносе времени клиенту отправляется письмо с прежним и новым временем записи (см. ФТ-008); при подтверждении/завершении/отмене отдельного уведомления клиенту не отправляется.
Выходные данные	Обновлённый статус записи либо обновлённое время записи

Таблица Б.16 – Оставление и модерация отзыва (ФТ-014)

Параметр	Описание
Наименование функции	Отзыв клиента о завершённой записи; публикация/скрытие отзыва администратором
Идентификатор	ФТ-014

Продолжение таблицы Б.16

Параметр	Описание
Цель и условие выполнения	Клиент оставляет отзыв после визита; администратор точки модерирует отзывы только о мастерах своей точки, владелец сети – обо всех мастерах сети
Входные данные	appointment_id, оценка rating (1–5), комментарий (опционально, до 2000 символов); для модерации – review_id, признак is_published
Требования к обработке	1) Проверка, что запись в статусе done и её клиент совпадает с текущим пользователем.
	2) Проверка, что отзыв на эту запись ещё не оставлен (уникальность appointment_id в review); при повторной попытке – ошибка 409.
	3) Текст комментария автоматически проверяется на нецензурную лексику и наличие ссылок/адресов сайтов – при обнаружении запрос отклоняется с соответствующим сообщением, отзыв не сохраняется.
	4) Отзыв публикуется сразу (is_published = true по умолчанию), без предварительной модерации; средний рейтинг мастера не хранится отдельно, а вычисляется по запросу как среднее по всем опубликованным отзывам мастера.
	5) Администратор/владелец может установить is_published = false для уже опубликованного отзыва – отзыв не удаляется, но не показывается на публичной странице мастера и не учитывается при расчёте рейтинга; повторная публикация возможна тем же действием в обратную сторону.
Выходные данные	Созданный отзыв

Таблица Б.17 – Формирование отчёта по выручке и загруженности мастеров (ФТ-015)

Параметр	Описание
Наименование функции	Отчёт по выручке, записям и загруженности мастеров за период
Идентификатор	ФТ-015
Цель и условие выполнения	Выполняется администратором точки (только своя точка) или владельцем сети (своя точка либо вся сеть на выбор) в разделе «Отчёты»
Входные данные	date_from, date_to; точка – неявно из роли администратора либо выбором владельца сети
Требования к обработке	1) За период вычисляются сводные показатели: общая выручка, общее число записей, средний чек, доля клиентов с более чем одной записью за период («повторные клиенты»).
	2) Строится график выручки по дням периода и разбивка числа записей по услугам.
	3) Для каждого мастера точки (сети) вычисляется: число записей, выручка, средний чек, средняя оценка по опубликованным отзывам.
	4) По кнопке «Экспорт Excel» тот же отчёт формируется файлом .xlsx с несколькими листами (сводка, выручка по дням, детализация по мастерам).
Выходные данные	Объект отчёта с перечисленными показателями и разбивками, пригодный для отображения на странице; либо файл .xlsx

Таблица Б.18 – Управление точками сети (ФТ-016)

Параметр	Описание
Наименование функции	Создание, редактирование и закрытие/открытие точки сети

Продолжение таблицы Б.18

Параметр	Описание
Идентификатор	ФТ-016
Цель и условие выполнения	Выполняется только владельцем сети в разделе «Точки сети»
Входные данные	Название, адрес, необязательный телефон, время открытия и закрытия, необязательное фото; для закрытия/открытия – признак is_active
Требования к обработке	1) Валидация: время закрытия позже времени открытия – иначе ошибка «close_time должен быть позже open_time».
	2) При создании точки короткое имя для URL (slug) генерируется сервером из названия автоматически, вручную не редактируется.
	3) Точка физически не удаляется – закрытие переводит её в is_active = false: точка пропадает с публичного сайта, но её мастера, записи и отчёты остаются доступны; открытие возвращает is_active = true.
	4) Часы работы точки – жёсткая граница: расписание мастеров этой точки (ФТ-011) и время записи клиента (ФТ-004, ФТ-005) не могут выходить за эти рамки; сужение часов работы уже назначенные смены мастеров и существующие записи не корректирует и не отменяет автоматически.
Выходные данные	Карточка точки; обновлённый список точек, используемый фильтром каталога мастеров и переключателем точки в панели администратора

5 Требования к видам обеспечения

5.1 Информационное обеспечение

5.1.1 Состав основных сущностей

Модель построена вокруг точки сети (salon): пользователь, мастер и запись ссылаются на конкретную точку, что обеспечивает разграничение видимости данных администратора его точкой (см. 3.1, 4.1.4). Профиль мастера (master) выделен в отдельную сущность и отделён от учётной записи пользователя (user_account) связью 1:0..1, поскольку не каждый пользователь является мастером. Помимо основных, бизнес-ориентированных сущностей, в модель включены служебные сущности, обеспечивающие безопасность: журнал попыток входа, токены сброса пароля и подтверждения email, сессии входа.

Правило генерации идентификаторов: все первичные ключи – UUID v4, генерируются на стороне приложения при создании записи (не автоинкремент – исключает угадывание/перечисление идентификаторов через API).

Основные сущности представлены в таблице Б.19.

Таблица Б.19 – Основные сущности системы и их атрибуты

Сущность	Основные атрибуты
salon (Точка сети)	id, name, slug, address, phone, open_time, close_time, photo_url, is_active, created_at
user_account (Пользователь)	id, first_name, last_name, phone, email, password_hash, vk_user_id, email_verified_at, role, salon_id (FK, только для роли «администратор»), is_blocked, token_version, deleted_at, created_at
master (Мастер)	id, user_id (FK → user_account), salon_id (FK → salon), specialization, photo_url, coefficient – множитель цены услуг мастера, is_active, deleted_at, created_at
service (Услуга)	id, name, description, price, duration_min, is_active, deleted_at

Продолжение таблицы Б.19

Сущность	Основные атрибуты
master_service (Услуга мастера)	master_id (FK), service_id (FK), price_override – необязательное точечное переопределение цены (если не задано, цена = services.price × masters.coefficient)
schedule (Расписание мастера)	id, master_id (FK), day_of_week (0–6), start_time, end_time, is_working, created_at
appointment (Запись)	id, client_id (FK → user_account), master_id (FK), service_id (FK), salon_id (FK), start_time, end_time, final_price, status, reminder_24h_sent_at, reminder_2h_sent_at, created_at
review (Отзыв)	id, appointment_id (FK, уникальный), client_id (FK), master_id (FK), service_id (FK), rating, comment, is_published, created_at
login_attempt_log (Попытка входа)	id, email_attempted, user_id (FK, может быть пуст), ip_address, success, created_at
password_reset_token (Токен сброса пароля)	id, user_id (FK), token_hash, expires_at, used_at, created_at
email_verification_token (Токен подтверждения email)	id, user_id (FK), token_hash, expires_at, used_at, created_at
user_session (Сессия входа)	id, user_id (FK), token_hash, expires_at, created_at
site_setting (Настройки сайта)	id, content (JSON: тексты, цветовая тема, шрифты, SEO), created_at

Во всех таблицах, хранящих токены (сброс пароля, подтверждение email, сессия), в базе данных сохраняется только SHA-256-хеш токена – сам токен существует только в ссылке/cookie на стороне пользователя и не

восстановим по содержимому базы данных.

5.1.2 Модель организации данных

Инфологическая модель разделена на две схемы для читаемости: рисунок Б.1 – сущности бизнес-домена (точки, пользователи, мастера, услуги, записи, отзывы); рисунок Б.2 – сущности, обеспечивающие безопасность входа, и настройки сайта. Точка сети (salon) – корневая сущность сети: на неё ссылаются мастер, администратор (через users.salon_id) и запись; профиль мастера отделён от учётной записи пользователя связью 1:0..1 (не каждый пользователь – мастер); услуга мастера (master_service) – связь M:N между мастером и услугой с необязательным переопределением цены.

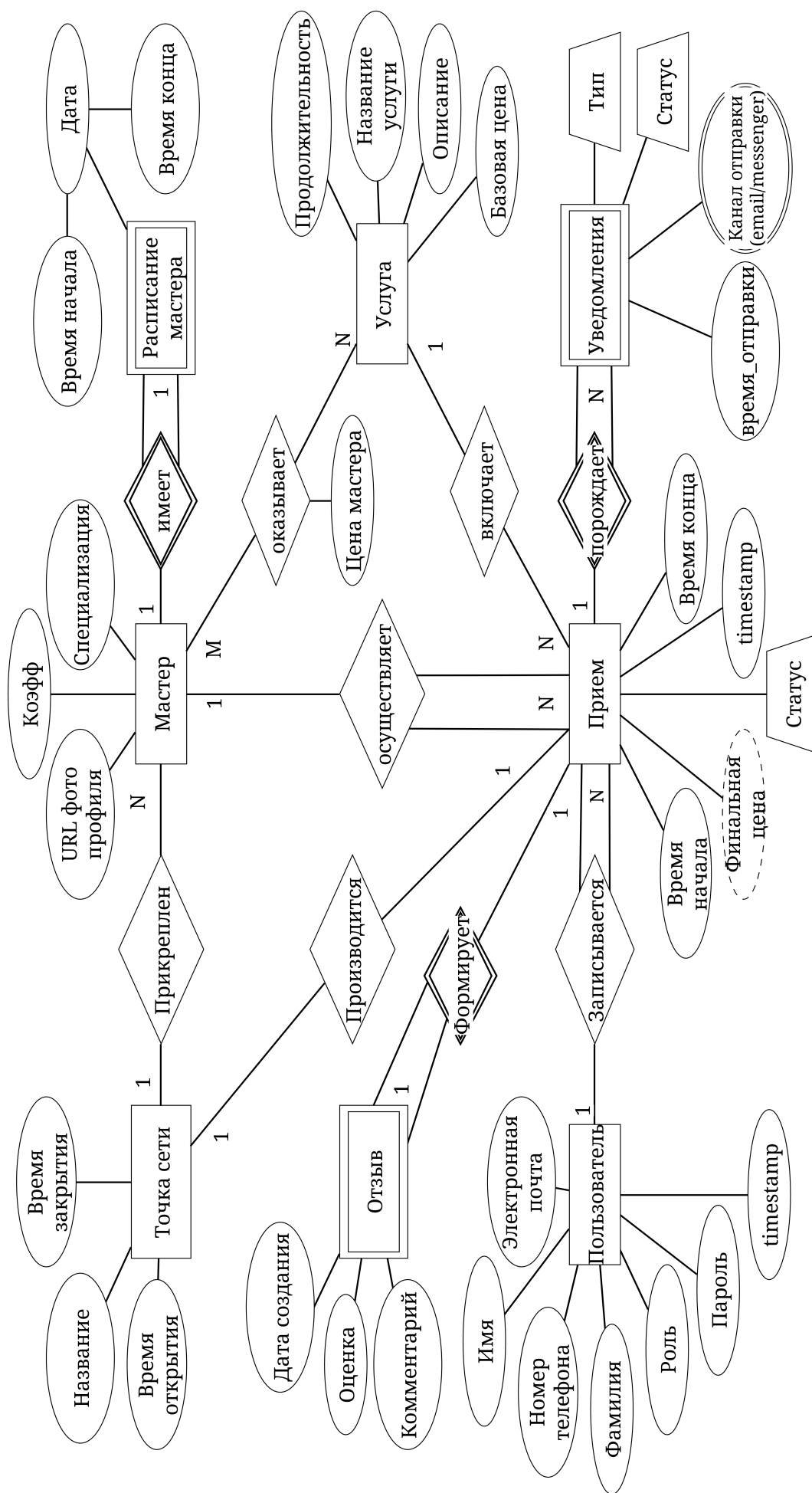


Рисунок Б.1 – Инфологическая модель – бизнес-домен

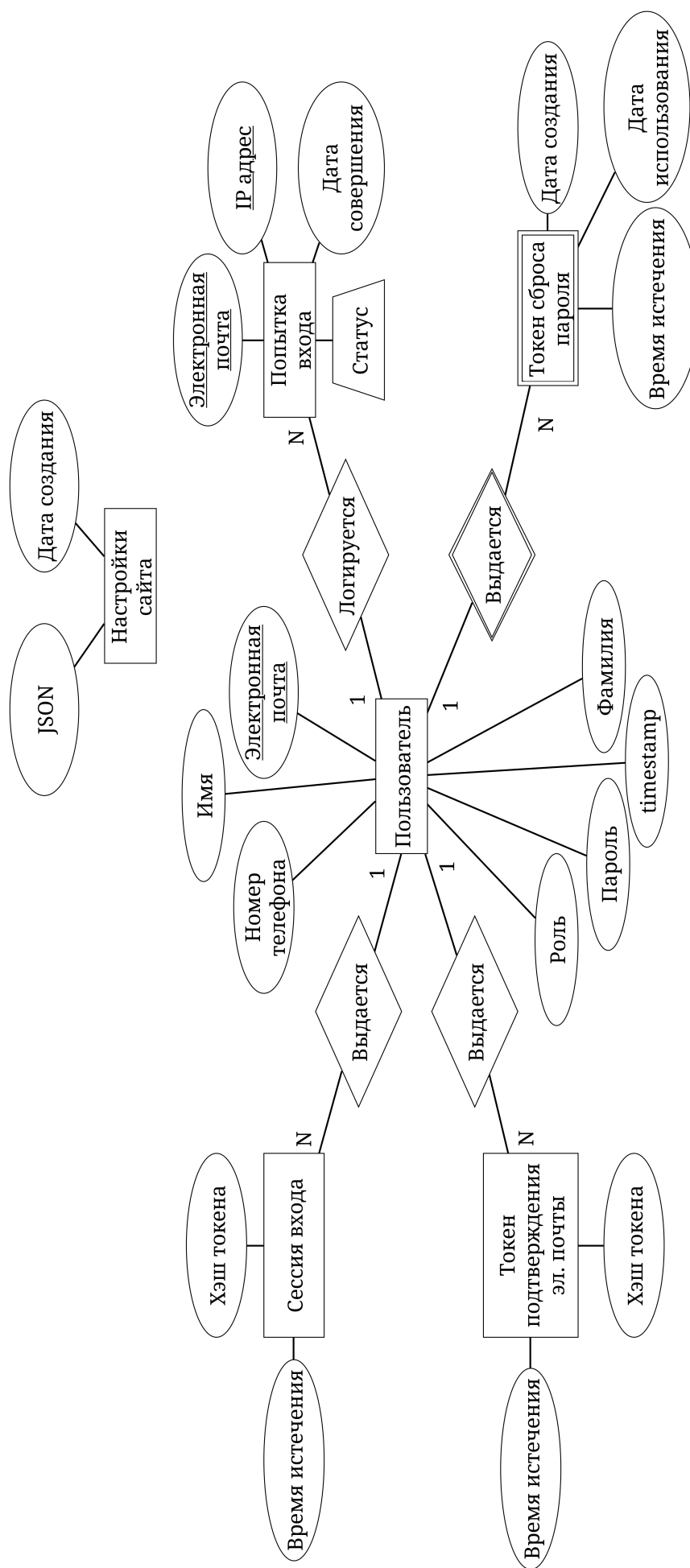


Рисунок Б.2 – Инфологическая модель – аутентификация и настройки сайта

Настройки сайта (пунктирная рамка на рисунке Б.2) – служебная сущность без связей с остальной моделью: это единственная строка конфигурации, не имеющая внешних ключей ни в одну другую таблицу и ни одной ссылающейся на неё.

Правила ссылочной целостности: прикладной код никогда не выполняет физическое (DELETE) удаление строк `user_account/master/service` – вместо этого используется мягкое удаление (поле `deleted_at`) или деактивация (`is_active = false`), история операций сохраняется. Ограничения внешних ключей на уровне БД – защитный рубеж на случай прямого изменения данных в обход приложения: удаление `user_account/master/service`, на которые ссылаются `appointment`, запрещено (ON DELETE RESTRICT); зависимые записи `master (schedule, master_service)`, `user_account (user_session, токены, отзывы)` и `appointment (review)` удаляются каскадно (ON DELETE CASCADE) только вместе со своим родителем.

5.1.3 Состав входной и выходной информации

Обязательность полей примеров ниже указана отдельным перечнем перед каждым примером.

Запись клиента к мастеру (все поля обязательны; клиент определяется из токена сессии, см. ФТ-005):

```
{
  "master_id": "b3f1...",
  "service_id": "a7c2...",
  "start_time": "2026-07-05T14:00:00+03:00"
}
```

Услуга каталога (обязательные поля: `name`, `price`, `duration_min`; `description` – опционально):

```
{
  "name": "Стрижка + борода",
  "description": "Классическая стрижка с оформлением бороды",
  "price": 2500,
  "duration_min": 60
}
```

Привязка услуги к мастеру, с необязательным переопределением цены (обязательно только service_id):

```
{
  "service_id": "a7c2...",
  "price_override": 3000
}
```

Выходная информация. Созданная запись:

```
{
  "id": "152e...",
  "status": "pending",
  "master": {"id": "b3f1...", "first_name": "Алексей", "last_name": "П."},
  "service": {"id": "a7c2...", "name": "Стрижка + борода", "price": 2500},
  "start_time": "2026-07-05T14:00:00+03:00",
  "end_time": "2026-07-05T15:00:00+03:00",
  "final_price": 3000
}
```

Отчёт по выручке за период (для администратора точки/владельца сети):

```
{
  "date_from": "2026-06-01",
  "date_to": "2026-06-30",
  "total_revenue": 385000.00,
  "total_appointments": 210,
  "avg_check": 1833.33,
  "repeat_clients_pct": 34.5,
  "revenue_by_day": [
    {"date": "2026-06-01", "revenue": 12500.00}
  ],
  "appointments_by_service": [
    {"service_name": "Стрижка + борода", "appointments": 72}
  ],
  "masters_breakdown": [
    {
      "master_name": "Алексей П.",
      "appointments": 58,

```

```

    "revenue": 145000.00,
    "avg_check": 2500.00,
    "avg_rating": 4.8
  }
]
}

```

5.1.4 Экранные формы

Экранные формы представлены на рисунках Б.3–Б.8. Рисунки представляют собой макеты экранных форм (wireframes), а не снимки экрана работающей системы.

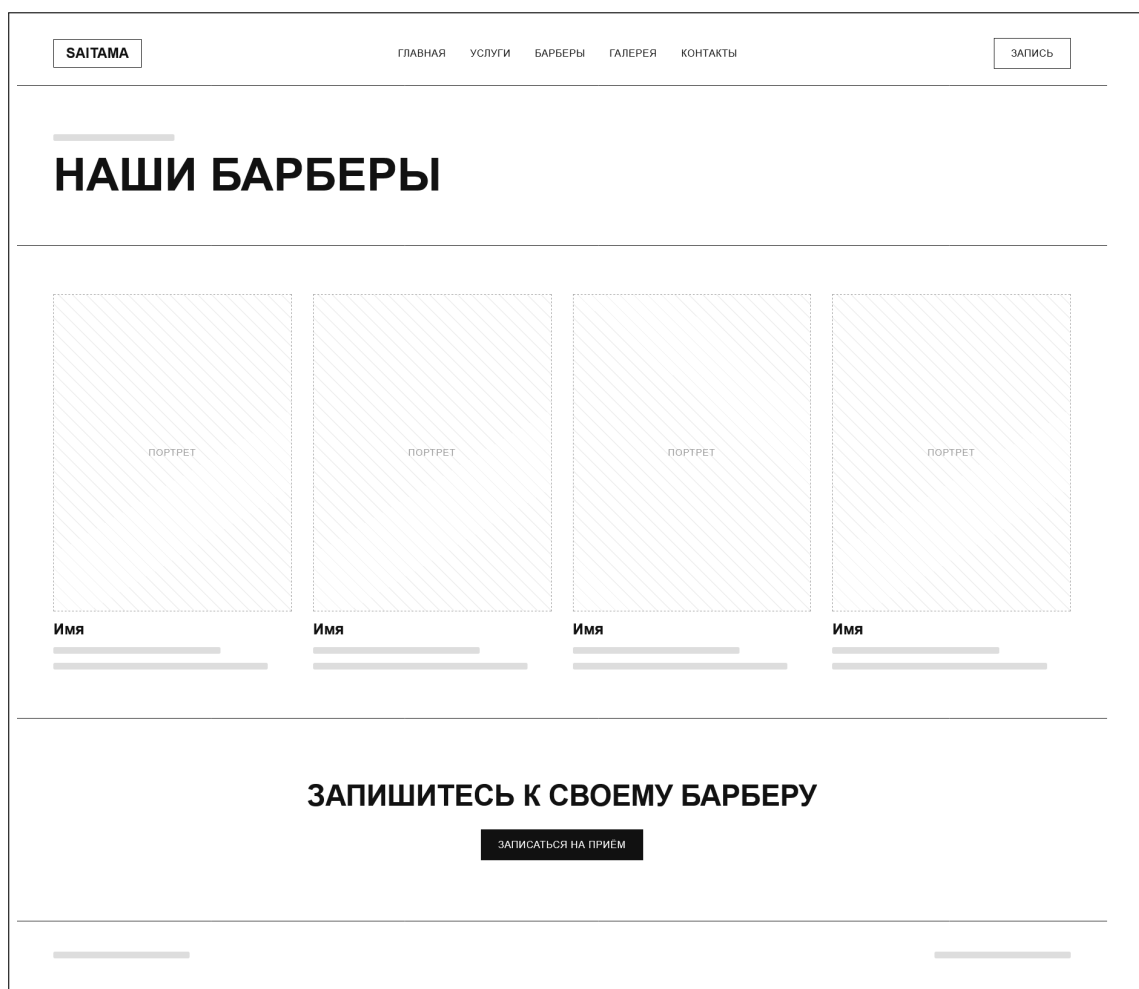


Рисунок Б.3 – Каталог мастеров (публичная часть сайта)



Рисунок Б.4 – Главная страница (публичная часть сайта)

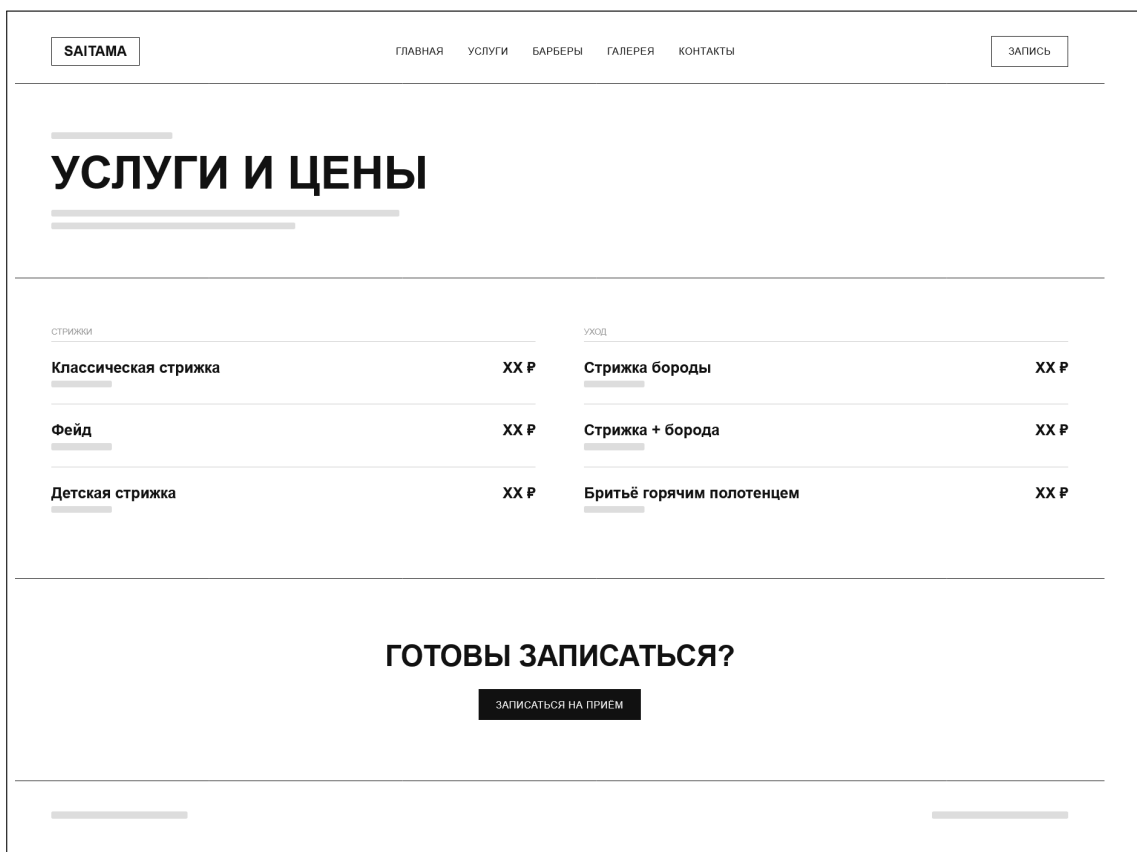


Рисунок Б.5 – Услуги и цены (публичная часть сайта)

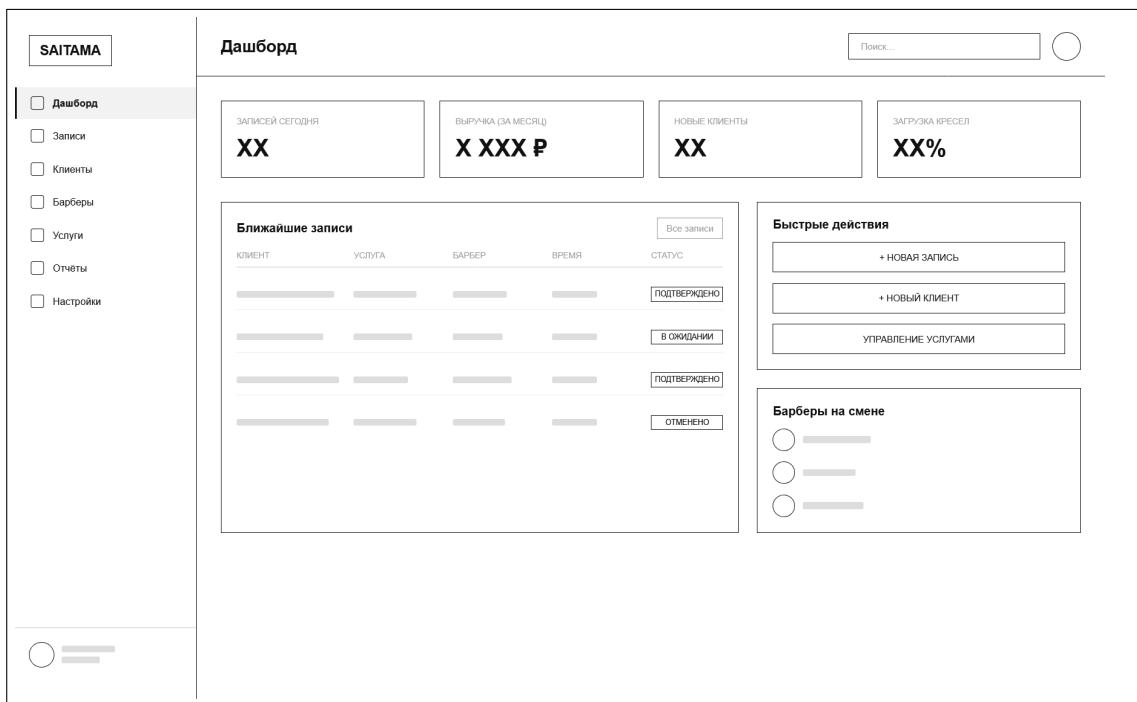


Рисунок Б.6 – Панель администратора – раздел «Дашборд» (статистика)

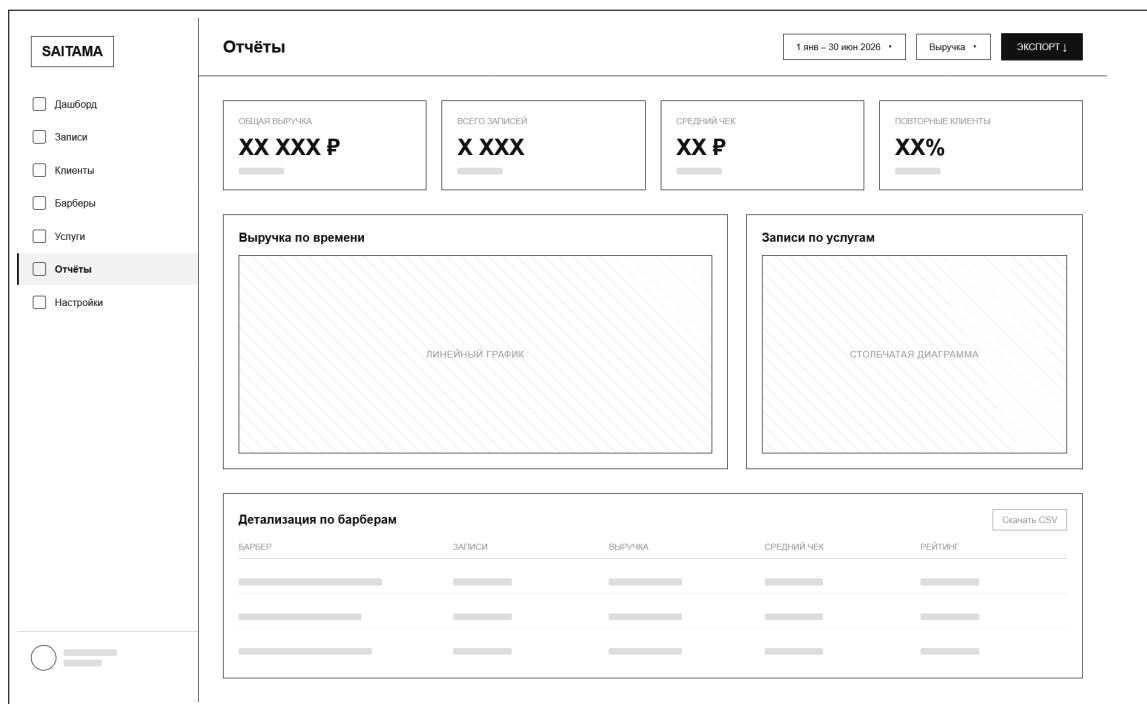


Рисунок Б.7 – Панель администратора – раздел «Отчёты»

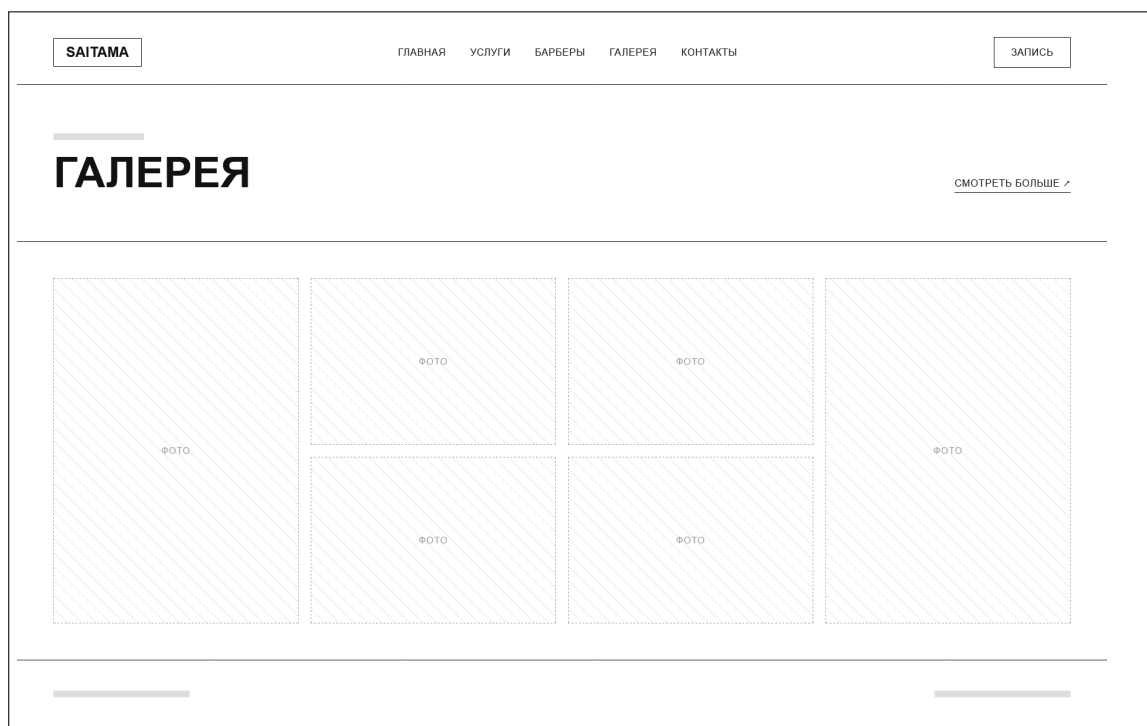


Рисунок Б.8 – Галерея (публичная часть сайта)

5.1.5 Классификаторы и кодовые системы

Классификатор и его кодовая система представлены в таблице Б.20. Отдельного классификатора категорий услуг система не ведёт – у услуги нет поля категории, каталог не группируется программно (см. 5.1.1). Каналов доставки писем в системе только один (email, см. 3.3 и ФТ-008), классификатор для одного значения не требуется.

Таблица Б.20 – Классификатор статусов записи

Код	Название	Описание
pending	Ожидает подтверждения	Запись создана, слот зарезервирован, ждёт подтверждения мастером/администратором
confirmed	Подтверждена	Мастер или администратор подтвердил запись
done	Завершена	Мастер отметил запись выполненной
cancelled	Отменена	Отменена клиентом, мастером или администратором

5.2 Лингвистическое обеспечение

Язык интерфейса: русский.

Основные термины: «Запись», «Слот», «Мастер», «Услуга», «Клиент», «Личный кабинет», «Напоминание».

Сокращения в пользовательском интерфейсе не допускаются; в документации допускаются общепринятые технические сокращения (API, JWT, ORM).

5.3 Программное обеспечение

Программное обеспечение, используемое системой, приведено в таблице Б.21.

Таблица Б.21 – Программное обеспечение

Компонент	Технология
Backend	Python 3.14, FastAPI, SQLAlchemy 2.0 (синхронный), Pydantic v2, Alembic
Frontend	Vue.js 3 (Composition API), Pinia, Tailwind CSS, Vite
База данных	PostgreSQL 16
Напоминания о записи	фоновый asyncio-цикл в процессе backend, без выделенного воркера (см. 4.1.1)
Аутентификация	JWT (python-jose), OAuth 2.1 с PKCE (VK ID)
Контейнеризация	Docker, docker-compose
Веб-сервер	Caddy (внешний обратный прокси: приём трафика на портах 80/443, TLS-терминация, автоматический сертификат Let's Encrypt); Nginx – внутри контейнера frontend: раздача собранного веб-приложения и внутреннее проксирование запросов /api/* на backend
Веб-аналитика	Яндекс.Метрика (счётчик подключён и активен, вынесен в отдельный файл /metrika.js)

5.4 Техническое обеспечение

Требования к техническому обеспечению приведены в таблице Б.22.

Таблица Б.22 – Необходимое техническое обеспечение

Ресурс	Требование
Сервер	VPS: от 2 CPU, 4 GB RAM, 40 GB SSD (обоснование ёмкости относительно нагрузочного теста – см. раздел 7)
Клиент	Браузер с поддержкой ES2020+: Chrome 90+, Firefox 85+, Safari 14+
Сеть	Постоянное интернет-соединение для клиента и сервера
Хранилище	Объектное хранилище или локальный диск для медиафайлов (логотип, фото мастеров, изображения услуг) – от 5 GB

6 Состав и содержание работ по созданию системы

Сумма длительностей этапов приведена в соответствии с датами начала и окончания работ (раздел 1.3) – 27 дней (22.06–18.07.2026). Этап «Интеграции» включает настройку фонового планировщика напоминаний (см. также раздел 5.3). Этапы «Тестирование» и «Развёртывание и сдача» объединены в один этап. Состав и содержание работ приведены в таблице Б.23.

Таблица Б.23 – Состав и содержание работ

№	Этап	Содержание	Срок
1	Проектирование	Обследование бизнес-процессов, проектирование схемы БД, диаграммы прецедентов	22.06– 25.06 (4 дня)
2	Разработка backend API	Реализация моделей, роутеров, авторизации (JWT, VK ID OAuth), логики слотов	26.06– 03.07 (8 дней)
3	Разработка frontend	Публичная часть (главная, запись, личный кабинет), административная панель	04.07– 10.07 (7 дней)
4	Интеграции	Настройка отправки email (SMTP), фонового планировщика напоминаний, интеграции с VK ID	11.07– 14.07 (4 дня)
5	Тестирование, развёртывание и сдача	Функциональное и нагрузочное тестирование, исправление ошибок, деплой на сервер, подготовка документации, демонстрация комиссии	15.07– 18.07 (4 дня)

7 Порядок контроля и приёмки системы

Виды, состав и методы испытаний

Функциональное тестирование:

- проверка всех функций ФТ-001–ФТ-016;
- тестирование граничных условий: одновременная запись двух клиентов на один слот (проверка EXCLUDE-ограничения); попытка отменить уже завершённую или ранее отменённую запись (проверка машины состояний, 4.2.1); попытка регистрации с уже существующим email;
- проверка корректности расчёта индивидуальных цен по мастерам.

Тестирование интерфейса:

- кроссбраузерное тестирование (Chrome, Firefox, Safari);
- проверка адаптивности на мобильных разрешениях (375–414 px).

Нагрузочное тестирование:

- одновременная работа не менее 20 пользователей с формой онлайн-записи без деградации времени отклика свыше 500 мс;
- проверка устойчивости EXCLUDE-ограничения при конкурентных запросах на один и тот же слот.

Обоснование технических требований к серверу (раздел 5.4) относительно нагрузочного теста: 20 одновременных пользователей формы записи создают, по оценке, не более 40 запросов/с к API в пике диалога выбора слота; при среднем времени ответа типового эндпоинта FastAPI менее 50 мс конфигурации 2 vCPU / 4 GB RAM достаточно с запасом для процесса backend и PostgreSQL – отдельного воркера, для которого требовался бы дополнительный запас, в системе нет (см. 4.1.1). Показатель оценочный и уточняется по фактическим результатам нагрузочного тестирования.

Критерии приёмки. Система считается прошедшей приёмочные испытания при одновременном выполнении условий:

- 1) все функции ФТ-001–ФТ-016 работают в соответствии с настоящим ТЗ;
- 2) не выявлено ни одного критического дефекта – делающего

невозможным выполнение сценариев регистрации/входа, онлайн-записи или административного управления записями;

3) число некритических дефектов не превышает 5 на каждые 100 проверенных тест-кейсов;

4) нагрузочное тестирование пройдено без деградации времени отклика свыше 500 мс (см. выше).

Порядок повторного тестирования. При обнаружении критического дефекта приёмочные испытания в затронутой области приостанавливаются; разработчик устраняет дефект в срок не более 2 рабочих дней; после устранения проводится регрессионное повторное тестирование только затронутого функционала (соответствующих тест-кейсов настоящего раздела), а не полного объёма испытаний заново. Полный повторный прогон требуется только при изменениях, затрагивающих более 30% проверенных сценариев.

8 Требования к документированию

Перечень документов

- техническое задание (данный документ);
- отчёт об обследовании бизнес-процессов (диаграммы прецедентов, физическая диаграмма, диаграммы действий, таблицы операций и документов);
- описание архитектуры системы и ER-диаграмма базы данных;
- программа и методика испытаний (ГОСТ РД 50-34.698-90) – определяет объём, порядок и критерии испытаний, детализируя раздел 7 настоящего ТЗ;
- документация API (OpenAPI/Swagger, автогенерируемая средствами FastAPI);
- руководство пользователя (для клиента и администратора);
- руководство администратора по развёртыванию (Docker, переменные окружения, миграции Alembic);
- отчёт о тестировании;
- исходный код с комментариями (репозиторий);
- презентация проекта для защиты.

Документ подготовлен на основе результатов исследования бизнес-процессов барбершопов и парикмахерских и охватывает требования, необходимые для проектирования и реализации системы.